



Magnetar: Threshold SLH- DSA via Reveal-and-Aggregate Seed Reconstruction (v0.1)

Zach Kelling

Lux Industries

2026

Abstract

We present *Magnetar v0.1*, a threshold construction for FIPS 205 SLH-DSA [43] that decomposes the SLH-DSA scheme seed across a (t, n) committee via byte-wise Shamir verifiable secret sharing over $\text{GF}(257)$, reconstructs the seed in aggregator memory during signing, and invokes the single-party FIPS 205 `SIGNDETERMINISTIC` on the reconstructed seed. The output is *byte-identical* to a centralized FIPS 205 signature on the same $(\text{SEED}, \mu, \text{ctx})$ triple — Magnetar’s *Class-N1-analog* claim, the hash-based counterpart of the Class N1 output-interchangeability property carried by Pulsar v0.1 [28] and Corona R-LWE v0.1 [30]. Any unmodified FIPS 205 verifier accepts a Magnetar signature without code change.

Honest claim. The v0.1 construction is straightforward and faithful to the underlying primitives. What it is *not* is a full threshold MPC over the SLH-DSA hash tree: the aggregator process holds the reconstructed master seed in memory for the duration of one COMBINE call. We make this caveat explicit and quantify the ~ 10 ms window during which the aggregator is in the trusted computing base (TCB) of the deployment. Standard mitigations (hardware-enclave aggregators, memory-encrypted VMs, ephemeral aggregator processes with `seccomp-bpf` and `forbidden-syscall` whitelists) neutralise the practical exposure; the paper documents both the residual risk and the operational hardening.

The companion artefact is `luxfi/magnetar v0.2.0` [37] (Tier B at time of writing; commits `55a7c96` through `db6c252`). The reference implementation totals $\sim 3\,992$ lines of Go across 12 production source files, backed by `cloudflare/circl/sign/slhdsa v1.6.3` [19]. The `TESTN1_BYTEEQUALITY` empirical gate exercises the byte-equality claim end-to-end across three thresholds; a passing run on M2 Pro completes in ~ 4.4 s at $(3, 2)$ and ~ 20 s at $(5, 3)$ under the production CGO build.

Magnetar is positioned alongside Pulsar (M-LWE) and Corona (R-LWE) as the *hash-based* leg of the Lux Polaris cert profile, providing cross-family cryptographic diversity against any single-family algorithmic break. The path to Tier A is mechanically structured: the v0.2 instantiation closes the aggregator-TCB caveat via a per-signature MPC over the SLH-DSA hash tree; the path to that v0.2 is sketched in §8 and *nontrivial* — SLH-DSA, unlike Schnorr or ML-DSA, has no linear share-aggregation identity that would admit a Komlo-style two-round threshold protocol.

Contents

1	Introduction	5
1.1	What Magnetar is	5
1.2	Why hash-based	6
1.3	What is novel	6
1.4	Trust model summary	7
1.5	What this paper specifies	7
2	Preliminaries	8
2.1	Notation	8
2.2	FIPS 205 SLH-DSA review	8
2.3	Shamir secret sharing over $\text{GF}(257)$	9
2.4	Threshold signature syntax	10
2.5	Security games	10
3	Construction	11
3.1	Hash-domain separation	11
3.2	DKG protocol	12
3.3	Threshold sign protocol	13
3.4	Combine (aggregator)	14
3.5	Verify (FIPS 205 passthrough)	14
3.6	Identifiable abort	14
3.7	Wire format	15

4	Security argument	15
4.1	Construction-equivalence (byte-equality)	15
4.2	Aggregator-honest tEUF-CMA reduction	16
4.3	The aggregator-honest assumption: honest disclosure	17
4.4	Side-channel posture	18
4.5	Nonce-uniqueness invariant	19
4.6	Composition with Polaris	19
4.7	Proof roadmap	19
5	Implementation	20
5.1	File layout	21
5.2	Dependencies	22
5.3	Build and test discipline	22
5.4	KAT vector format	22
5.5	Configuration-as-data	23
5.6	Concurrency posture	23
5.7	Lux Polaris integration	23
6	Evaluation	24
6.1	Parameter sets	24
6.2	KAT determinism	24
6.3	Per-signature latency	24
6.4	Wire bandwidth	25
6.5	Empirical claims verified	26
7	Comparison to Pulsar and Corona	26
7.1	The shared v0.1 trust caveat	26
7.2	Mathematical independence	27
7.3	Performance comparison	27
7.4	Lifecycle composition	28
7.5	Trust caveat: what differs across lanes	28
7.6	Where each lane is the right answer	28
8	Path to full MPC threshold (v0.2 research direction)	29
8.1	Why Komlo-style two-round threshold does not apply	29
8.2	Candidate MPC topologies	29
8.3	Suggested v0.2 direction	30
8.4	What v0.2 will <i>not</i> solve	30
8.5	Adjacent literature	31
9	Related work	31
9.1	Threshold hash-based signatures	31
9.2	Lattice-family threshold signatures	32
9.3	Schnorr-family threshold (FROST)	33
9.4	Threshold ECDSA (CGGMP21)	33
9.5	BLS aggregate	33
9.6	Standards landscape	33
9.7	Comparison table	34
9.8	Where Magnetar’s contribution lives	34

10 Conclusion	35
10.1 What Magnetar v0.2.0 is	35
10.2 What Magnetar v0.2.0 is not	35
10.3 Path to Tier A	36
10.4 Closing	36

1 Introduction

Post-quantum threshold signatures are the load-bearing primitive of any blockchain consensus design that wants to survive a cryptographically relevant quantum adversary. The classical workhorses — BLS aggregate [15, 12] and ECDSA with CGGMP21 [17] — fail catastrophically under Shor’s algorithm, and the threshold replacements available for the post-quantum era do not all ground their security in the same underlying mathematical assumption. Defense in depth at the cryptographic primitive layer requires not just “a PQ scheme,” but a portfolio of PQ schemes whose security assumptions are *algorithmically independent*: a break of any one family does not propagate to the others.

The Lux Polaris cert profile [35] encodes this discipline in production. Polaris is the maximum-assurance Quasar finality profile that pairs three independent threshold lanes:

- **Pulsar** — threshold Module-LWE [28, 14], the lattice lane in $\mathbb{Z}_q[X]/(X^{256} + 1)$ with a 48-bit prime;
- **Corona** — threshold Ring-LWE [30, 14], a structurally distinct lattice instantiation that does not share Pulsar’s module parameter choice; and
- **Magnetar** (this paper) — threshold FIPS 205 SLH-DSA [43], the hash-based lane whose security rests on SHA-3 / Keccak collision resistance and preimage resistance rather than on any structured lattice assumption.

The three legs are deliberately drawn from *different mathematical families*: a polynomial-time algorithm against module-lattice problems would break Pulsar and Corona simultaneously but leave Magnetar intact; a polynomial-time preimage attack on Keccak would break Magnetar but leave the lattice lanes intact; a single quantum oracle that defeats both families is, on current evidence, an inconsistent ask. Polaris finality requires *all three* lanes to verify, so an adversary must break the conjunction — a strictly harder task than breaking any single primitive.

Magnetar is the third leg. This paper specifies the construction at github.com/luxfi/magnetar v0.2.0 (Tier B; “library available, formal proofs and independent audit on the Tier A roadmap”), the security argument under the v0.1 trust caveat, and the path to Tier A.

1.1 What Magnetar is

Magnetar v0.1 is a threshold signature scheme for FIPS 205 SLH-DSA [43]. SLH-DSA is the NIST hash-based signature standard (formerly SPHINCS+ [8]); it is stateless, requires no trapdoor or structured ring, and inherits its security from the standard properties of cryptographic hash functions (collision resistance, preimage resistance, second-preimage resistance, and the Boneh–Shoup [13] sub-exponential preimage bound under quantum adversaries).

The Magnetar construction departs structurally from the lattice-family threshold templates of FROST [33, 20] or Pulsar [28]. SLH-DSA has *no linear share-aggregation identity*: there is no algebraic operation analogous to FROST’s $z = \sum_i z_i$ that produces a valid SLH-DSA signature from per-party contributions on the same message. A direct MPC over the SLH-DSA hash tree (the v0.2 research target of §8) is possible in principle but unsubtle in practice: it would require distributed generation of every WOTS+ hash chain and every Merkle authentication path, with cost proportional to the entire SLH-DSA signing transcript ($\sim 10^4$ hash invocations for SHAKE-192s).

Magnetar v0.1 takes a different route. We share the *SLH-DSA scheme seed* (the input to FIPS 205 KEYGEN / DERIVEKEY), not the resulting signature. The scheme seed is a 96-byte buffer (192s/192f) or 128-byte buffer (256s); it is the unique secret input from which the full SLH-DSA secret key, public key, and every signature derive deterministically. Sharing the seed byte-wise via Shamir over GF(257) admits the standard Lagrange reconstruction at $x = 0$ to recover the seed inside the aggregator; the aggregator then invokes the single-party FIPS 205 SIGNDERMINISTIC on the reconstructed seed and zeroizes the secret material before return.

The honest characterisation: *the cryptographic primitive is FIPS 205 SLH-DSA; the threshold layer is a byte-wise Shamir VSS over the scheme seed; the trust model is reveal-and-aggregate*. The aggregator process is in the TCB for the brief window during which the seed is reconstructed in memory. This is the same trust model Pulsar v0.1 and Corona v0.1 ship, and is documented operationally in `DEPLOYMENT-RUNBOOK.md`.

1.2 Why hash-based

Lattice cryptography (Module-LWE, Ring-LWE, NTRU) is the consensus post-quantum direction for performance reasons — ML-KEM [41], ML-DSA [42], and FALCON [25] all operate in milliseconds, ship signatures in the few-KB range, and parallelise well. But the lattice family rests on a single set of related-problem assumptions (Module-LWE, Ring-LWE, NTRU, Module-SIS) whose hardness is conjectured but whose ultimate security margin is the subject of ongoing cryptanalytic refinement. The cryptanalytic surface includes BKZ block-size cost models [6, 34], dual sieves [1], structured-lattice algebraic attacks on cyclotomic rings, and quantum-amplified sieves whose constants remain in modest motion. A polynomial-time algorithm for any of these problems would invalidate the entire lattice signature ecosystem simultaneously.

Hash-based signatures rest on assumptions that are *older, more broadly studied, and structurally orthogonal*. FIPS 205 SLH-DSA’s security reduces to:

- collision resistance of the hash function family (SHA-3 / SHAKE for the FIPS 205 SHAKE modes Magnetar targets);
- preimage resistance of the same;
- the absence of unforeseen structural weaknesses in the WOTS+ and FORS one-time / few-time signature schemes that build on the hash function.

These properties have been studied for decades. SHA-3 in particular emerged from a multi-year open competition [9]; its internal structure (Keccak’s sponge construction over a permutation) is qualitatively distinct from Merkle–Damgård, BLAKE, and the keyed-permutation Romulus and Ascon families. A quantum attack that defeats SHA-3 is one that defeats hash functions *as a class*, a strictly stronger ask than defeating any one family.

The trade-off is performance: SLH-DSA signatures are 16–36 KB, signing takes hundreds of milliseconds on commodity hardware, and the construction is the slowest of the FIPS-standardised PQ schemes. For a *single* hot-path signature this would be a non-starter. For the *Polaris third lane* it is precisely the trade-off we want: Polaris’s BLS lane sets the latency floor, Pulsar sets the per-block PQ floor, and Magnetar shoulders the diversity insurance — it does not need to be the fastest, only to be *independent* of the other two.

1.3 What is novel

The v0.1 cryptographic primitives are not new. Shamir [49] introduced byte-wise secret sharing over a small prime in 1979; the construction has been textbook material since the 1980s [27]. FIPS 205 [43] is the NIST-standardised SLH-DSA. The reveal-and-aggregate threshold pattern (“share the seed, reconstruct in memory, call the centralised primitive”) is itself an obvious construction and has been described in the threshold-cryptography folklore for ages.

What **luxfi/magnetar** contributes is the *composition*:

1. A Class-N1-analog claim and its empirical validation (TESTN1_BYTEEQUALITY, end-to-end byte equality between the threshold and centralised paths across three (n, t) configurations).
2. A KAT-deterministic test discipline: every secret-bearing component (Shamir polynomial, mask, commit) is derivable from a single 48-byte master seed via SHA-256 chaining, so the full submission-grade KAT package is reproducible bit-identically from a clean checkout.
3. Hash-function domain separation across every primitive — cSHAKE256 with function name “Magnetar” and per-purpose customisation strings (MAGNETAR-DKG-COMMIT-V1, MAGNETAR-SIGN-R1-V1,

MAGNETAR-SEED-SHARE-V1, etc.) so cross-context replay attacks reduce to cSHAKE256 collision resistance.

4. Honest operational disclosure: the v0.1 reveal-and-aggregate trust caveat is documented at the construction-specification level, at the deployment-runbook level, and at the proof-claim level. No covert reliance on aggregator integrity; no hidden trust assumptions.
5. A Polaris cert-profile composition: Magnetar joins Pulsar and Corona as the third lane of the maximum-assurance profile, sharing the LSS lifecycle adapter contract [48] and the Prism transcript-binding discipline [28].

1.4 Trust model summary

We state the trust model at the outset; it is the dominant honest caveat of v0.1.

In the TCB at sign time: the aggregator process. The aggregator reconstructs the master seed S in memory at step 6 of COMBINE (§3.4), invokes SIGNDETERMINISTIC, and zeroizes S before return. During the $\sim 1\text{--}50$ ms window between reconstruction and zeroization (SLH-DSA SHAKE-192s signing latency on M2 Pro is on the order of 10 ms), an adversary with arbitrary code-execution rights on the aggregator host can extract S via the standard kernel-level vectors (`/proc/<pid>/mem`, `ptrace`, `coredumps`, `swap-file` leaks).

NOT in the TCB: a passive network observer (Round-1 commits hide r_i, s'_i behind cSHAKE256; v0.1 envelopes are plaintext, which v0.2 closes via ML-KEM-768 wrapping). A *single* Byzantine committee member (forging requires reconstructing S , which requires t shares). A Byzantine aggregator that has not received valid Round-2 reveals (the commit-bind check in COMBINE step 1a rejects tampered reveals).

Mitigations. The aggregator should run in a memory-encrypted VM (AMD SEV-SNP, Intel TDX), with `coredumps` and `ptrace` disabled, `swap` off, on dedicated hardware. The aggregator process should be ephemeral: `fork()` per signature, `exit` after COMBINE returns. `DEPLOYMENT-RUNBOOK.md` §“Aggregator hardening” lists the production checklist.

Tier A path. v0.2 (research target; §8) replaces the reveal-and-aggregate aggregator with a per-signature MPC over the SLH-DSA hash tree. This eliminates the aggregator-TCB caveat at the cost of substantially higher per-signature communication. The construction is nontrivial because SLH-DSA has no linear share-aggregation identity; the v0.2 design surveys the candidate MPC topologies in §8.2.

1.5 What this paper specifies

Section 2 reviews FIPS 205 SLH-DSA, byte-wise Shamir secret sharing over $\text{GF}(257)$, the threshold-signature syntax, and the security games. Section 3 gives the byte-exact v0.1 reveal-and-aggregate construction in pseudocode, keyed to the Go canonical at [ref/go/pkg/magnetar/](https://github.com/secure-systems-lab/magnetar) byte-for-byte. Section 4 states the security argument: construction-equivalence to centralized FIPS 205 SLH-DSA, the UF-CMA reduction under the aggregator-honest assumption, and the proof roadmap for the open obligations. Section 5 surveys the reference implementation (3 992 LOC, 12 files). Section 6 reports parameter sets and per-signature latency from the production test suite. Section 7 locates Magnetar alongside Pulsar and Corona in the Polaris cert profile. Section 8 sketches the path to a Tier A v0.2 that closes the aggregator-TCB caveat via a per-signature MPC. Section 9 surveys the (sparse) threshold-SLH-DSA literature and the broader threshold-hash-based-signature work. Section 10 restates the honest Tier B status and the Tier A path.

Tier B status. At the time of writing, Magnetar carries *Tier B* status in the Lux Lyttle tier discipline: production library available, KAT vectors deterministic, byte-equality empirically validated, but *no* mechanised proofs and *no* independent cryptographer audit have landed. The path to Tier A (mechanised proof artefacts, independent review sign-off) is sketched in BLOCKERS.md BLK-7 and BLK-9 and in §4.7 below. *Use Magnetar today as a library at your operational risk*; in the Lux Polaris production deployment it is one of three independent lanes, so a Magnetar break is insurance lost, not safety lost.

2 Preliminaries

2.1 Notation

- λ — security parameter (set to 192 for SHAKE-192s, 256 for SHAKE-256s).
- n — committee size, $1 \leq n \leq 256$.
- t — reconstruction threshold, $1 \leq t \leq n$.
- $\text{GF}(p)$ — finite field of order p ; throughout this paper, $p = 257$ is the smallest prime greater than 255.
- `SEED_SIZE` — byte-length of the SLH-DSA scheme seed: 96 bytes for SHAKE-192s and SHAKE-192f; 128 bytes for SHAKE-256s. This is the secret input to FIPS 205 `DERIVEKEY`.
- P_i — party i , $1 \leq i \leq n$. Each party is identified by a 32-byte `NodeID`.
- $x_i \in [1, 256]$ — party P_i ’s Shamir evaluation point, distinct per party.
- $c_i \in \{0, 1\}^{8 \cdot \text{SEED_SIZE}}$ — party P_i ’s DKG contribution.
- $S \in \{0, 1\}^{8 \cdot \text{SEED_SIZE}}$ — joint master SLH-DSA seed.
- $\text{share}_i \in [0, 257]^{\text{SEED_SIZE}}$ — party P_i ’s Shamir share of the master byte-sum (per byte position, modulo $p = 257$).
- (pk, sk) — the FIPS 205 SLH-DSA public/private keypair derived from S via FIPS 205 `KEYGEN`.
- σ — a FIPS 205 SLH-DSA signature (the wire bytes).
- μ — the message being signed; ctx — the FIPS 205 context string (0–255 bytes).
- $sid \in \{0, 1\}^{128}$ — a session identifier binding one threshold-sign attempt across the network.
- $\kappa \in \mathbb{N}$ — session attempt counter (for replay isolation between retries).
- `cSHAKE256` — the `cSHAKE256` customisable hash from NIST SP 800-185 [31], with function-name string “Magnetar” and per-context customisation tags.
- $\parallel$ — byte concatenation; $\oplus$ — bitwise XOR.

2.2 FIPS 205 SLH-DSA review

FIPS 205 [43] (formerly SPHINCS+ [8]) is the NIST stateless hash-based signature standard. The Magnetar reference implementation targets the FIPS 205 SHAKE parameter sets:

Table 1: FIPS 205 SHAKE parameter sets exposed by Magnetar v0.1. Values from FIPS 205 [43] §10.1 Table 2.

Parameter set	NIST PQ Cat.	n (WOTS+)	h	d	$ pk $	$ \sigma $
SLH-DSA-SHAKE-192s	3	24 B	63	7	48 B	16 224 B
SLH-DSA-SHAKE-192f	3	24 B	66	22	48 B	35 664 B
SLH-DSA-SHAKE-256s	5	32 B	64	8	64 B	29 792 B

The FIPS 205 construction is built from the following layered components, top to bottom:

- **XMSS-HT (hypertree)**: a tree of d XMSS layers. The leaves of the root layer are XMSS public keys; the leaves of the bottom layer are FORS public keys. Every layer has height h/d .

- **XMSS (eXtended Merkle Signature Scheme)**: a binary Merkle tree where each leaf is a WOTS+ public key.
- **WOTS+ (Winternitz One-Time Signature)**: a one-time signature whose security reduces to hash function preimage and second-preimage resistance.
- **FORS (Forest of Random Subsets)**: the few-time signature scheme used at the bottom of the hypertree.
- **Tweakable hash F, H, T_ℓ** : the cryptographic primitives, instantiated as SHAKE256 for the SHAKE-family Magnetar targets.

Key generation. `KEYGEN` [43] §9.1 samples a $3n$ -byte secret triple $(SK.seed, SK.prf, PK.seed)$ and computes $PK.root = T_h(SK.seed, PK.seed, ADRS)$ as the XMSS-HT root. The public key is $pk = (PK.seed, PK.root)$; the secret key is $sk = (SK.seed, SK.prf, PK.seed, PK.root)$. The total scheme-seed size (the input to `DERIVEKEY`) is $4n$ bytes: 96 for $n = 24$ (192s/192f) and 128 for $n = 32$ (256s).

Deterministic signing. `SIGNdet`(sk, μ, ctx) [43] §9.2 computes a randomiser $R \leftarrow \text{PRF}_{SK.prf}(ctx || \mu)$ and proceeds deterministically. The output is byte-identical across calls with the same (sk, μ, ctx) . This determinism is the property that makes the Magnetar Class-N1-analog claim trivially provable: if two parties reconstruct the same sk and call `SIGNdet` on the same (μ, ctx) , they obtain byte-identical signatures.

Randomised signing. `SIGNrand`(sk, rng, μ, ctx) [43] §9.2 samples a fresh n -byte randomiser from rng . Magnetar’s threshold-sign path uses the deterministic variant by default (so the byte-equality claim holds); a *randomized* flag exposes the randomised variant for callers that want hedge against fault-injection on the *PRF* derivation.

Verification. `VERIFY`(pk, μ, σ, ctx) [43] §9.3 walks the XMSS-HT root, recomputes $PK.root$ from the FORS and XMSS authentication paths in σ , and accepts iff it matches the $PK.root$ embedded in pk . The verifier consumes no secret material; *any unmodified FIPS 205 verifier accepts a Magnetar signature with no code change*.

2.3 Shamir secret sharing over GF(257)

We instantiate Shamir secret sharing [49] byte-wise over GF(p) with $p = 257$. The choice of $p = 257$ is dictated by two constraints:

1. $p > 255$, so every byte value $0, \dots, 255$ is a distinct field element.
2. $p < 2^9$, so each share value fits in a 9-bit lane, packed as one `uint16` per byte position on the wire.

The trade-off: $p = 257$ admits at most $p - 1 = 256$ distinct non-zero evaluation points, capping the committee size at 256 (constant `MaxCommittee257` in `params.go`). For the Lux validator committees this is comfortably above the production thresholds (Quasar operates at $K \leq 21$ [35]).

Sharing. To share a `SEED_SIZE`-byte secret $s \in [0, 256]^{\text{SEED_SIZE}}$ across n parties at threshold t :

1. For each byte position $b \in [0, \text{SEED_SIZE})$, sample $t - 1$ random coefficients $a_{b,1}, \dots, a_{b,t-1} \in \text{GF}(257)$ and define $f_b(x) = s[b] + \sum_{j=1}^{t-1} a_{b,j} x^j \pmod{257}$.
2. For each recipient P_i at evaluation point $x_i = i$ (one-indexed), compute $share_i[b] = f_b(x_i) \pmod{257}$ for all b .
3. Wire-form: pack each $share_i$ as a sequence of `SEED_SIZE` big-endian `uint16` values ($2 \cdot \text{SEED_SIZE}$ bytes total).

Reconstruction. Given any t shares $\{(x_i, share_i)\}_{i \in Q}$ with $|Q| \geq t$, the constant term of each f_b is recovered by Lagrange interpolation at $x = 0$:

$$s[b] = \sum_{i \in Q} \lambda_i^Q(0) \cdot share_i[b] \pmod{257}, \quad \lambda_i^Q(0) = \prod_{j \in Q, j \neq i} \frac{-x_j}{x_i - x_j} \pmod{257}. \quad (1)$$

The Lagrange coefficients $\lambda_i^Q(0)$ are computed once per quorum using Fermat’s little theorem for modular inversion ($a^{-1} \equiv a^{p-2} \pmod{p}$); see `shamir.go` lines 148–164.

Information-theoretic privacy. For any $|Q| < t$, the joint distribution of $\{share_i\}_{i \in Q}$ is uniformly distributed over $[0, 257)^{|Q| \cdot \text{SEED_SIZE}}$ independent of $\mathbf{s}$. This is the standard Shamir privacy property [49, 27]: every $t - 1$ -subset of shares reveals zero information about the secret, as a matter of information theory (not computational complexity). This property is the load-bearing secrecy guarantee of Magnetar v0.1 *against the committee* — a coalition of $t - 1$ or fewer committee members has no useful information about S .

The GF(257)/byte-domain mismatch. The Lagrange reconstruction (1) produces a value in $[0, 257)$; the SLH-DSA scheme seed is byte-valued in $[0, 256)$. The single value 256 has no byte representation. Magnetar handles this *constructively*: the Shamir reconstruction is fed into a cSHAKE256 mix

$$S = \text{cSHAKE256}(\text{byteSum}_{BE} \parallel \text{committee_root}, N = \text{“Magnetar”}, S = \text{“MAGNETAR-SEED-SHARE-V1”}, 8). \quad (2)$$

which absorbs the GF(257)-valued `byteSum` as a $2 \cdot \text{SEED_SIZE}$ -byte big-endian buffer and emits a SEED_SIZE -byte S . The cSHAKE256 mix domain-separates the seed from raw Shamir output and gives a constant-length seed regardless of the value distribution of `byteSum`.

2.4 Threshold signature syntax

We use the standard threshold-signature syntax of Canetti [16] and Boneh–Shoup [13] as specialised to SLH-DSA.

Definition 1 (Magnetar threshold signature scheme). Π_{Magnetar} is a tuple of probabilistic polynomial-time algorithms (DKG , SIGN , COMBINE , VERIFY):

- $\text{DKG}(1^\lambda, n, t, \text{Committee})$: a three-round interactive protocol among committee parties. Outputs the joint group public key pk , per-party key shares $\{(NodeID_i, x_i, share_i)\}_{i=1}^n$, and a transcript hash τ_{DKG} .
- $\text{SIGN}_i(share_i, sid, \kappa, \text{Quorum}, \mu)$: an interactive 2-round subprotocol executed by party P_i in Quorum . Outputs a Round-1 commit message and a Round-2 reveal message.
- $\text{COMBINE}(pk, \mu, ctx, sid, \kappa, \text{Quorum}, t, \{R1_i, R2_i\}_{i \in \text{Quorum}}, \{share_j\}_{j=1}^n)$: a deterministic function executed by the aggregator. Outputs a FIPS 205 SLH-DSA signature σ or $\perp$.
- $\text{VERIFY}(pk, \mu, \sigma, ctx)$: the unmodified FIPS 205 verifier of [43] §9.3.

The deliberate choice that VERIFY is the unmodified FIPS 205 verifier — not a Magnetar-specific envelope — is the load-bearing operational property: deployment shipping a Magnetar threshold signer needs to add no new verification code on the consuming side. Every SLH-DSA verifier in the wild (the FIPS 205 reference, openssl, cloudflare/circl, libsignal) accepts Magnetar signatures with no change.

2.5 Security games

We adopt the standard threshold unforgeability game (cf. Pulsar [28] §5).

Definition 2 (Threshold EUF-CMA). $\text{Game}_{\Pi}^{\text{tEUF-CMA}}(\mathcal{A}, \lambda)$:

1. **Setup.** The challenger runs DKG with the adversary $\mathcal{A}$ playing the corrupt parties. $\mathcal{A}$ commits to a corrupt set F with $|F| < t$ at the outset (static corruption model). The challenger gives $\mathcal{A}$ the joint pk and $\{\text{share}_i : i \in F\}$.
 2. **Queries.** $\mathcal{A}$ adaptively requests signatures on messages $\mu_1, \dots, \mu_{Q_S}$. The challenger runs the honest threshold sign protocol and returns each σ_i . $\mathcal{A}$ may invoke the random oracle $\mathcal{O}_H$ up to Q_H times.
 3. **Forgery.** $\mathcal{A}$ outputs (μ^*, σ^*) with $\mu^* \notin \{\mu_i\}$, and wins iff $\text{VERIFY}(pk, \mu^*, \sigma^*, \text{ctx}) = 1$.
- The advantage is $\text{Adv}_{\Pi, \mathcal{A}}^{\text{tEUF-CMA}}(\lambda) = \Pr[\mathcal{A} \text{ wins}]$. Π is (Q_S, Q_H, T, ϵ) -tEUF-CMA secure if for every $\mathcal{A}$ within (Q_S, Q_H, T) , $\text{Adv} \leq \epsilon$.

The aggregator-honest variant of this game (Magnetar’s v0.1 security target) restricts $\mathcal{A}$ from compromising the aggregator: the aggregator is assumed to execute COMBINE honestly and zeroize secret material as specified.

Definition 3 (Aggregator-honest tEUF-CMA). The aggregator-honest variant $\text{Game}_{\Pi}^{\text{tEUF-CMA-AH}}(\mathcal{A}, \lambda)$ is identical to Definition 2 except that the corrupt set F is constrained to exclude the aggregator process and the adversary has no access to the master seed S or the reconstructed sk during their in-memory lifetime.

The v0.1 security claim (Theorem 3 below) is stated in the aggregator-honest model. The honest disclosure is that any deployment must operationally enforce *aggregator-honest* via the mitigations of §1.4 or accept the residual risk of an aggregator compromise. v0.2 (§8) drops this assumption.

3 Construction

This section specifies the Magnetar v0.1 reveal-and-aggregate construction byte-for-byte. The pseudocode is keyed to the Go canonical at [ref/go/pkg/magnetar/](https://github.com/ethereum/magnetar) [37]; line references are to commit db6c252 (Tier B graduation).

3.1 Hash-domain separation

Every cryptographic hash invocation in Magnetar is cSHAKE256 [31] with function-name string "Magnetar" and a per-purpose customisation string. The seven canonical tags are listed in Table 2.

Table 2: Magnetar v0.1 cSHAKE256 domain separation. Every secret-bearing hash invocation is bound to exactly one tag; cross-tag replay reduces to cSHAKE256 collision resistance.

Tag	Purpose
MAGNETAR-DKG-COMMIT-V1	committee-root digest (sorted committee binding)
MAGNETAR-DKG-TRANSCRIPT-V1	DKG transcript digest (Round-2 equivocation check)
MAGNETAR-SIGN-R1-V1	threshold-sign Round-1 commit D_i
MAGNETAR-SIGN-R1-MAC-V1	(reserved for v0.2 per-pair MAC layer)
MAGNETAR-SIGN-MASK-V1	per-attempt mask r_i derivation
MAGNETAR-SEED-SHARE-V1	Shamir coefficient stream + final seed mix

The protocol-wide domain separation context is the byte string "lux-magnetar-v0.1" (`tagDomainSep`). It is prepended to the DKG transcript binding so that a separate Magnetar version (or a different Lux PQ primitive that happened to reuse the same hash function) cannot collide.

3.2 DKG protocol

The DKG is a three-round broadcast-channel protocol over the canonical sorted committee. Round 1 distributes Shamir-shared dealer contributions; Round 2 commits each party to its local view of the ordered envelope set; Round 3 detects equivocation and produces the final key shares.

Round 1 (dealer P_i , `dkg.go:137`).

1. Sample $c_i \xleftarrow{\$} \{0, 1\}^{8 \cdot \text{SEED_SIZE}}$ from *rng*.
2. Sample $2(t-1) \cdot \text{SEED_SIZE}$ bytes of coefficient material from *rng*. If short, stretch via `cSHAKE256(·, S = "MAGNETAR-SEED-SHARE-V1")`.
3. For each byte position $b \in [0, \text{SEED_SIZE})$, define the polynomial $f_{b,i}(x) = c_i[b] + \sum_{j=1}^{t-1} a_{b,j,i} \cdot x^j \pmod{257}$ with the $a_{b,j,i}$ unpacked from the coefficient stream (2 bytes per coefficient, taken modulo 257).
4. For each recipient P_k (committee order, $k = 1, \dots, n$), compute $\text{share}_{i \rightarrow k}[b] = f_{b,i}(x_k) \pmod{257}$ for every b via Horner's method (`shamir.go:111`).
5. Construct the envelope $\text{Env}_{i \rightarrow k} = (\text{share}_{i \rightarrow k}, c_i)$ for every k . Note: the dealer's full c_i is duplicated into every envelope — this is the v0.1 reveal-and-aggregate compromise (every committee member learns every other dealer's contribution). v0.2 closes this by wrapping each envelope under ML-KEM-768 keyed to the recipient's long-term identity public key (BLOCKERS.md CR-8).
6. Broadcast $\text{DKG_ROUND1MSG}_i = (\text{NodeID}_i, \{\text{Env}_{i \rightarrow k}\}_{k=1}^n)$.

Round 2 (party P_i , `dkg.go:185`).

1. Validate every received DKG_ROUND1MSG_j contains an envelope for every committee member (no omissions), with the expected wire sizes for every share and every contribution. Reject malformed messages with `ERRDKGMISSINGENVELOPE`, `ERRDKGSHARESIZE`, or `ERRDKGCONTRIBSIZE`.
2. Compute the transcript digest:

$$\begin{aligned} \tau_{\text{DKG}} &= \text{"lux-magnetar-v0.1"} \parallel \text{Mode} \parallel t_{\text{LE}} \parallel n_{\text{LE}} \parallel \text{NodeID}_1 \parallel \dots \parallel \text{NodeID}_n \\ D &= \text{cSHAKE256}(\tau_{\text{DKG}} \parallel \text{for_each_dealer}_j(\text{for_each_recipient}_k(\text{share}_{j \rightarrow k} \parallel c_j)), \\ &\quad \text{N} = \text{"Magnetar"}, S = \text{"MAGNETAR-DKG-TRANSCRIPT-V1"}, 32 \text{ B}). \end{aligned}$$

3. Broadcast $\text{DKG_ROUND2MSG}_i = (\text{NodeID}_i, D)$.

Round 3 (party P_i , `dkg.go:278`).

1. Validate every $\text{DKG_ROUND2MSG}_j.D = D$ (computed locally). A mismatch is an *identifiable abort* — `ABORTEVIDENCEEquivocation` names the offending dealer P_j and carries the conflicting digest pair as on-chain slashing evidence.
2. Compute the per-party share of the master byte-sum mySumShare_i by adding the per-byte shares received from every dealer: $\text{mySumShare}_i[b] = \sum_{j=1}^n \text{share}_{j \rightarrow i}[b] \pmod{257}$, for $b \in [0, \text{SEED_SIZE})$.
3. Compute the master byte-sum (also derivable as the sum of every dealer's contribution): $\text{masterByteSum}[b] = \sum_{j=1}^n c_j[b] \pmod{257}$, for $b \in [0, \text{SEED_SIZE})$.
4. Compute the committee root:

$$\text{committee_root} = \text{cSHAKE256}(\text{"MAGNETAR-COMMITTEE-V1"} \parallel \text{sorted_committee}, \text{N} = \text{"Magnetar"}, S = \text{"MAGNETAR-COMMITTEE-V1"}, 32 \text{ B}).$$

The committee is canonically sorted ascending by *NodeID* (`dkg.go:423`).

5. Mix to the master seed via equation (2): $S = \text{cSHAKE256}(\text{masterByteSum}_{\text{BE}} \parallel \text{committee_root}, \text{N} = \text{"Magnetar"}, S = \text{"MAGNETAR-SEED-SHARE-V1"}, \text{SEED_SIZE} \text{ B})$.

6. Derive the SLH-DSA secret key: $(sk, pk) \leftarrow \text{CIRCL.SLHDSA.SCHEME}(ID).\text{DERIVEKEY}(S)$.
Verify the resulting public key matches what every other honest party would compute (this is automatic: every party has identical *masterByteSum* and *committee_root* inputs).
Discard sk ; publish pk as the joint group public key.
7. Compute the chain-pinned transcript hash binding $(\text{committee_root}, pk, t)$: $\tau_{\text{out}} = \text{TRANSCRIPTHASH}(\text{"MA"}$
8. Output: $\text{KEYSHARE}_i = (\text{NodeID}_i, x_i = i, \text{Share} = \text{mySumShare}_i, \text{Pub} = pk, \text{Mode})$ and τ_{out} .
9. Zeroize all intermediate buffers: S , the recovered sk , mySumShare_i wire-form, masterByteSum , the mix input, c_i , and every per-byte share $f_{b,i}(x_k)$ (ZEROIZEBYTES / ZEROIZEU16SLICE / ZEROIZEPRIVATEKEY).

DKG correctness. For any quorum $Q \subseteq [n]$ with $|Q| \geq t$, Lagrange interpolation of $\{(x_i, \text{mySumShare}_i)\}_{i \in Q}$ at $x = 0$ recovers *masterByteSum*:

$$\sum_{i \in Q} \lambda_i^Q(0) \cdot \text{mySumShare}_i[b] = \sum_{i \in Q} \lambda_i^Q(0) \cdot \sum_{j=1}^n f_{b,j}(x_i) = \sum_{j=1}^n \underbrace{\sum_{i \in Q} \lambda_i^Q(0) \cdot f_{b,j}(x_i)}_{=c_j[b]} = \text{masterByteSum}[b], \quad (3)$$

where the inner Lagrange sum recovers $f_{b,j}(0) = c_j[b]$ by the standard Shamir reconstruction property. The cSHAKE256 mix (2) is deterministic, so the reconstructed master seed is identical to the master seed computed at DKG Round 3. $\square$

3.3 Threshold sign protocol

The sign protocol is a two-round commit-and-reveal between a quorum $T \subseteq [n]$ of size t .

Round 1 (signer P_i for $i \in T$, `threshold.go:150`).

1. Sample raw entropy $\text{rngBytes} \xleftarrow{\$} \{0,1\}^{16 \cdot \text{SEED_SIZE}}$ from *rng*.
2. Derive per-attempt mask via cSHAKE256:
 $r_i = \text{cSHAKE256}(\text{rngBytes} \parallel \text{sid} \parallel \kappa_{BE} \parallel \text{NodeID}_i, \text{N} = \text{"Magnetar"}, \text{S} = \text{"MAGNETAR-SIGN-MASK-V1"})$
The mask-mix construction ensures that even a deterministic *rng* that emits the same bytes across two attempts produces distinct masks per attempt (the κ_{BE} field rules out cross-attempt collision).
3. Compute the byte-wise XOR-masked share $\text{masked}_i[b] = \text{share}_i[b] \oplus r_i[b]$ for $b \in [0, 2 \cdot \text{SEED_SIZE})$.
4. Build the Round-1 transcript τ_1 binding $(\text{sid}, \kappa, \text{Quorum}, \text{NodeID}_i, pk, \mu)$ in NIST SP 800-185 encoded form (`threshold.go:230`):
 $\tau_1 = \text{LEFTENCODE}(|\text{parts}|) \parallel \text{ENCODESTRING}(\text{sid}) \parallel \text{ENCODESTRING}(\kappa) \parallel \dots \parallel \text{ENCODESTRING}(\mu)$.
5. Compute the commit:
 $D_i = \text{cSHAKE256}(r_i \parallel \text{masked}_i \parallel \tau_1, \text{N} = \text{"Magnetar"}, \text{S} = \text{"MAGNETAR-SIGN-R1-V1"}, 32 \text{ B})$.
6. Broadcast $\text{ROUND1MESSAGE}_i = (\text{NodeID}_i, \text{sid}, \kappa, D_i)$.
7. Retain $r_i, \text{masked}_i, \tau_1$ locally for Round 2 reveal. Wipe *rngBytes* and the mask-mix input.

Round 2 (signer P_i , `threshold.go:198`).

1. Validate received Round-1 messages: every ROUND1MESSAGE_j must carry the same (sid, κ) as the local state. Reject cross-session messages with `ERRSESSIONMISMATCH`, cross-attempt with `ERRATTEMPTMISMATCH`.
2. Construct the reveal $\text{PARTIALSIG}_i = r_i \parallel \text{masked}_i$ ($4 \cdot \text{SEED_SIZE}$ bytes).
3. Broadcast $\text{ROUND2MESSAGE}_i = (\text{NodeID}_i, \text{sid}, \kappa, \text{PARTIALSIG}_i)$.

3.4 Combine (aggregator)

COMBINE is a deterministic pure function executed by the aggregator (any party in the quorum, or an external aggregator with read access to the share directory). `combine.go:47`.

Inputs: $(pk, \mu, ctx, randomized, sid, \kappa, \text{Quorum}, t, \{R1_i\}, \{R2_i\}, \{\text{KEYSHARE}_j\}_{j=1}^n)$.

Steps:

1. Index $\{R1_i\}$ by sender NodeID_i . Reject cross-session messages.
2. For each $R2_i$ in the quorum:
 - (a) Parse $r_i \parallel \text{masked}_i = \text{PARTIALSIG}_i$.
 - (b) Re-derive $D'_i = \text{cSHAKE256}(r_i \parallel \text{masked}_i \parallel \tau_{1,i}, N = \text{"Magnetar"}, S = \text{"MAGNETAR-SIGN-R1-V1"},$ with $\tau_{1,i}$ reconstructed from public inputs.
 - (c) Compare D'_i against $R1_i.D_i$ in constant time (`ctEqual32`). On mismatch: reject with `ERRROUND2COMMITBAD`.
 - (d) Recover $\text{share}_i = \text{masked}_i \oplus r_i$.
3. Pair each recovered share with the matching `KEYSHARE` entry's x_i (the aggregator gets the directory of $\{\text{KEYSHARE}_j\}_{j=1}^n$ so it can recover evaluation points).
4. Run Lagrange reconstruction over $\text{GF}(257)$ at $x = 0$ per equation (1) on the first t recovered shares, yielding $\text{reconstructedByteSum} \in [0, 257]^{\text{SEED_SIZE}}$.
5. Compute committee_root from the full directory of $\{\text{KEYSHARE}_j\}_{j=1}^n$ (the same definition as DKG Round 3 step 4).
6. Reconstruct the master seed via equation (2): $S = \text{cSHAKE256}(\text{reconstructedByteSum}_{BE} \parallel \text{committee_root}, N = \text{"Magnetar"}, S = \text{"MAGNETAR-SEED-SHARE-V1"}, \text{SEED_SIZE B})$.
7. Derive the centralized FIPS 205 secret key: $(sk_{\text{rec}}, pk_{\text{rec}}) \leftarrow \text{CIRCL.SLHDSA.SCHEME}(ID).\text{DERIVEKEY}(S)$.
8. Verify $pk_{\text{rec}} = pk$. On mismatch: zeroize secret material and return `ERRPUBKEYMISMATCH`.
9. Compute the FIPS 205 signature: $\sigma = \begin{cases} \text{SLHDSA.SIGNDETERMINISTIC}(sk_{\text{rec}}, \mu, ctx) & \text{if } randomized = \text{false} \\ \text{SLHDSA.SIGNRANDOMIZED}(sk_{\text{rec}}, rng, \mu, ctx) & \text{if } randomized = \text{true} \end{cases}$
10. Zeroize S , sk_{rec} , the mix input, $\text{reconstructedByteSum}$, and the recovered $\{\text{share}_i\}$. Return σ .

Determinism and byte equality. With $randomized = \text{false}$ (the default), the FIPS 205 `SIGNDETERMINISTIC` output is a deterministic function of $(sk_{\text{rec}}, \mu, ctx)$. Since sk_{rec} is itself a deterministic function of S , and S is a deterministic function of the share set (independent of which quorum of size $\geq t$ is used), every quorum produces the same σ . This is the empirical content of `TESTN1_BYTEEQUALITY_DIFFERENTQUORUMSSAMESIGNATURE (§??)`.

3.5 Verify (FIPS 205 passthrough)

`VERIFY verify.go:32` is a single dispatch to `CIRCL.SLHDSA.PUBLICKEY.UNMARSHALBINARY(pk.Bytes)` followed by `SLHDSA.VERIFY(pk, NEWMESSAGE( $\mu$ ),  $\sigma$ ,  $ctx$ )`. *No Magnetar-specific envelope or wrapper*. This is the Class-N1-analog property: the produced signatures live in the exact FIPS 205 signature space and any FIPS 205 verifier accepts them unmodified.

3.6 Identifiable abort

The v0.1 protocol commits to identifiable abort: every detected misbehaviour produces verifiable on-chain evidence suitable for slashing.

Table 3: Identifiable abort taxonomy in Magnetar v0.1.

Complaint	Detected at	Evidence
COMPLAINTEQUIVOCATION	DKG Round 3	$(digest_{\text{honest}}, digest_{\text{accused}})$
COMPLAINTBADDELIVERY	DKG Round 2	malformed envelope (parsed bytes)
ERRROUND2COMMITBAD	COMBINE step 2c	$(R1_i, R2_i)$ pair
ERRPUBKEYMISMATCH	COMBINE step 8	quorum + share set

The v0.1 protocol *omits* the per-pair MAC layer that Pulsar uses to authenticate Round 1 against tampering by a network attacker (BLOCKERS.md CR-7). The omission is documented; v0.2 adds the MAC layer once the trust model tightens to include passive-network-observer secrecy of envelopes.

3.7 Wire format

The wire formats are byte-exact specifications that gate cross-implementation interoperability. Magnetar’s wire bytes are defined by the Go canonical’s serialisation in `types.go` and must match across any future port (C++, Rust, hardware).

Table 4: Magnetar v0.1 wire-format sizes. “Conceptual size” is the serialised payload; the actual envelope adds an SP 800-185 leftEncode length prefix per field.

Object	Size (bytes)
ROUND1MESSAGE commit D_i	32
ROUND2MESSAGE partial sig $r_i masked_i$	$4 \cdot \text{SEED_SIZE}$ (e.g., 384 for SHAKE-192s)
KEYSHARE share buffer	$2 \cdot \text{SEED_SIZE}$
DKGSHAREENVELOPE $(share, c)$	$3 \cdot \text{SEED_SIZE}$
SIGNATURE.BYTES (SHAKE-192s)	16 224
SIGNATURE.BYTES (SHAKE-192f)	35 664
SIGNATURE.BYTES (SHAKE-256s)	29 792

The signature wire format is *exactly* the FIPS 205 wire format (per FIPS 205 §9.4); there is no Magnetar prefix, suffix, or envelope.

4 Security argument

This section states the v0.1 security claims and the proof roadmap. We state two theorems: (i) construction-equivalence to centralized FIPS 205 SLH-DSA (the Class-N1-analog property), and (ii) tEUF-CMA in the aggregator-honest model under FIPS 205 SLH-DSA’s UF-CMA assumption.

We are honest about what is *not* proved here: a full mechanised proof of either claim has not been built. The proof roadmap is in §4.7; this paper states the claims with informal-but-rigorous arguments and points at the planned mechanised companion.

4.1 Construction-equivalence (byte-equality)

Theorem 1 (Class-N1-analog byte equality). *Fix a parameter set $MODE \in \{\text{SHAKE-192s}, \text{SHAKE-192f}, \text{SHAKE-256s}\}$ and let (n, t) be a valid threshold configuration. Let DKG complete honestly, producing a committee Committee, joint public key pk , and share set $\{\text{KEYSHARE}_j\}_{j=1}^n$. Let S be the master seed computed at DKG Round 3 (step 5 of §3.2).*

For any message μ , context ctx , session identifier sid , attempt counter κ , and quorum $Q \subseteq [n]$ with $|Q| = t$, if every party in Q executes the threshold sign protocol of §3.3 honestly and the aggregator executes COMBINE of §3.4 honestly with $randomized = false$, then

$$COMBINE(pk, \mu, ctx, false, sid, \kappa, Quorum, t, \{R1_i, R2_i\}_{i \in Q}, \{KEYSHARE_j\}_{j=1}^n) = SLHDSA.SIGNDETERMINISTIC(\mu, ctx, S) \quad (4)$$

Proof. By inspection of the protocol.

Step 1: Quorum-independence of seed reconstruction. (3) shows that any quorum Q of size $\geq t$ reconstructs the same *masterByteSum* from $\{mySumShare_i\}_{i \in Q}$ via Lagrange interpolation. By construction, COMBINE step 4 performs exactly this Lagrange interpolation on the shares recovered from the Round-2 reveals (each $share_i$ recovered as $masked_i \oplus r_i$). Since the cSHAKE256 mix (2) is a deterministic function of $(masterByteSum, committee_root)$, and $committee_root$ is computed from the same sorted committee in both DKG and COMBINE, the reconstructed master seed S_{rec} in COMBINE step 6 equals the DKG master seed S .

Step 2: Pubkey consistency. COMBINE step 7 computes $(sk_{rec}, pk_{rec}) \leftarrow SLHDSA.DERIVEKEY(S_{rec})$. Since $S_{rec} = S$ (step 1), and FIPS 205 DERIVEKEY is a deterministic function of its seed input, $pk_{rec} = pk$ (step 8 of COMBINE verifies this equality and aborts otherwise) and sk_{rec} is byte-identical to the sk that a centralized signer would obtain.

Step 3: Signature determinism. COMBINE step 9 invokes $SLHDSA.SIGNDETERMINISTIC(sk_{rec}, \mu, ctx)$. Per FIPS 205 §9.2, SIGNDETERMINISTIC is a deterministic function of (sk, μ, ctx) . The output signature bytes are byte-identical to those a centralized signer with seed S would produce on the same (μ, ctx) . $\square$ $\square$

Empirical validation. The byte-equality theorem is empirically validated by TESTN1_BYTEEQUALITY_TH_MATCHESCENTRALIZED [37], which runs the threshold protocol end-to-end against the centralized FIPS 205 path and asserts BYTES.EQUAL on the signature outputs. The test covers $(n, t) \in \{(3, 2), (5, 3), (7, 4)\}$ when not running under *-short*; the short-form CI run executes only the (3, 2) configuration (~ 4.4 s on M2 Pro).

A complementary test TESTN1_BYTEEQUALITY_DIFFERENTQUORUMSSAMESIGNATURE exercises the quorum-independence corollary: two *distinct* quorums $Q_A = \{0, 1, 2\}$ and $Q_B = \{2, 3, 4\}$ over the same DKG output produce *byte-identical* signatures on the same message. This is the direct consequence of Theorem 1’s quorum-independence step.

Corollary 2 (FIPS 205 verifier acceptance). *A Magnetar v0.1 signature produced by COMBINE (with randomized = false) is accepted by any unmodified FIPS 205 SLH-DSA verifier under the joint group public key pk.*

Proof. By Theorem 1, the Magnetar signature bytes equal those of a centralized SIGNDETERMINISTIC(DERIVEKEY(S), μ , ctx). Centralized FIPS 205 signatures verify under their derived public key per FIPS 205 §9.3. $\square$ $\square$

4.2 Aggregator-honest tEUF-CMA reduction

Theorem 3 (Magnetar v0.1 tEUF-CMA, aggregator-honest model). *Let $\Pi_{Magnetar}$ be the v0.1 construction of §3 instantiated at parameter set $MODE \in \{SHAKE-192s, SHAKE-192f, SHAKE-256s\}$. Under*

- the UF-CMA security of FIPS 205 SLH-DSA at the chosen parameter set [43, 8];
- the collision resistance of cSHAKE256 at output length 32 B (i.e., 2^{128} -bit collision security);
- the PRF security of cSHAKE256 with a high-entropy key;
- the standard Shamir secret-sharing information-theoretic privacy property over $GF(257)$ for any $(t-1)$ -subset [49];

in the aggregator-honest model (Definition 3), with adversary $\mathcal{A}$ corrupting up to $t-1$ committee members and the network channel,

$$\text{Adv}_{\Pi_{\text{Magnetar}}, \mathcal{A}}^{\text{tEUF-CMA-AH}}(\lambda) \leq \text{Adv}_{\text{SLH-DSA}, \mathcal{B}}^{\text{UF-CMA}}(\lambda) + \frac{Q_H + Q_S}{2^{256}} + \frac{Q_S}{2^{128}}, \quad (5)$$

where $\mathcal{B}$ is a black-box reduction running $\mathcal{A}$ with Q_S signing queries and Q_H random-oracle queries, with running time within $\text{poly}(\lambda)$ of $\mathcal{A}$.

Proof sketch. We construct a reduction $\mathcal{B}$ that converts a Magnetar forger $\mathcal{A}$ into an SLH-DSA forger.

Setup. $\mathcal{B}$ receives an SLH-DSA UF-CMA challenge pk . $\mathcal{B}$ runs the Magnetar DKG with $\mathcal{A}$ playing the corrupt parties; $\mathcal{B}$ programs the DKG so that the joint group public key equals the challenge pk . The programming exploits Shamir’s information-theoretic privacy: for $|F| < t$, $\mathcal{A}$ ’s view of the shares $\{\text{share}_i\}_{i \in F}$ is statistically uniform regardless of the master seed ([49, 27] Theorem 3.6), so $\mathcal{B}$ can simulate the corrupt parties’ shares as uniform random and the honest parties’ contributions as random strings consistent with the joint pk . This produces a perfect simulation of the honest DKG view modulo the negligible cSHAKE256 collision probability on the transcript binding ($\leq Q_H/2^{256}$).

Signing oracle. For each adversary signing query μ_i , $\mathcal{B}$ forwards μ_i to its SLH-DSA UF-CMA signing oracle and obtains a signature $\sigma_i = \text{SIGNDETERMINISTIC}(sk, \mu_i, \text{ctx})$. $\mathcal{B}$ then simulates the threshold protocol transcript by:

1. Choosing a fresh (sid_i, κ_i) .
2. For each honest party in the quorum, sampling a fresh mask r_i uniformly and computing $\text{masked}_i = \text{share}_i \oplus r_i$ as the protocol prescribes.
3. For the corrupt parties’ contributions in the quorum, using the values $\mathcal{A}$ supplies; if those values cause a commit-bind check failure in COMBINE, $\mathcal{B}$ aborts the simulation as the protocol does.
4. Returning σ_i from its SLH-DSA oracle as the Magnetar signature. By Theorem 1, this is byte-identical to what COMBINE would produce on an honest run of this protocol given that the reconstructed seed equals the DKG-output seed (which equals the challenge sk ’s seed by the DKG programming).

The simulation is perfect except for: (i) a $Q_S/2^{128}$ failure probability arising from sid collisions (the simulator samples sid_i from $\{0, 1\}^{128}$); (ii) a $(Q_H + Q_S)/2^{256}$ failure probability for cSHAKE256 collisions on the transcript hashes $\tau_1, \tau_{\text{DKG}}$.

Extraction. If $\mathcal{A}$ produces a successful Magnetar forgery (μ^*, σ^*) with $\text{VERIFY}(pk, \mu^*, \sigma^*, \text{ctx}) = 1$, then by Corollary 2 σ^* is a valid FIPS 205 signature under pk . $\mathcal{B}$ outputs (μ^*, σ^*) as its UF-CMA forgery.

Validity. $\mathcal{B}$ wins the SLH-DSA UF-CMA game iff $\mu^* \notin \{\mu_i\}$, which is exactly $\mathcal{A}$ ’s winning condition. The reduction loses the simulation failure probability, giving (5). $\square$ $\square$

Provable security level. $\text{Adv}_{\text{SLH-DSA}}^{\text{UF-CMA}}$ at the SHAKE-192s parameter set is upper-bounded by $\sim 2^{-192}$ classical and $\sim 2^{-128}$ quantum (Grover); at SHAKE-256s, $\sim 2^{-256}$ classical and $\sim 2^{-128}$ quantum. The $Q_S/2^{128}$ term in (5) dominates at the quantum bound once Q_S approaches 2^{60} ; this is a non-tight reduction (the session-id collision term could be tightened via a stateful sid enforcement that the consensus layer provides) but adequate at the operational scale of Lux Polaris (Q_S per epoch $\leq 10^6$, well below the binding term).

4.3 The aggregator-honest assumption: honest disclosure

Theorem 3 is stated in the aggregator-honest model. This is the honest v0.1 trust caveat. The caveat covers exactly the in-memory lifetime of the reconstructed master seed S inside COMBINE:

In the TCB during Combine steps 6–10:

- the master seed S (length `SEED_SIZE`);
- the recovered SLH-DSA secret key sk_{rec} (FIPS 205 wire form);
- the cSHAKE256 mix input $mixInput$ (length $2 \cdot \text{SEED_SIZE} + 32$);
- the reconstructed byte-sum vector $reconstructedByteSum$ (length $2 \cdot \text{SEED_SIZE}$);
- the per-quorum recovered shares $\{share_i\}_{i \in Q}$.

Threat surface. An adversary with arbitrary code-execution rights on the aggregator host during this window can exfiltrate S via:

- `/proc/<pid>/mem` read access (requires same UID or `CAP_SYS_PTRACE`);
- ptrace-based debugging (same capability);
- coredumps written to disk (requires `kernel.core_pattern` misconfiguration or signal-triggered crash);
- swap-file or hibernation image leaks (requires swap enabled or sleep-mode triggered);
- a malicious kernel or hypervisor (requires platform-level compromise).

Operational hardening. `DEPLOYMENT-RUNBOOK.md` §“Aggregator hardening” enumerates the production checklist:

1. Run the aggregator in a memory-encrypted VM (AMD SEV-SNP or Intel TDX) with attestation; verify attestation on every signing session.
2. Disable coredumps (`ulimit -c 0`, `kernel.core_pattern` pointing to `/dev/null`).
3. Disable swap (`swapoff -a`); disable hibernation (`systemctl mask hibernate.target sleep.target`).
4. Disable ptrace (`kernel.yama.ptrace_scope = 3`); bind the aggregator process under `PR_SET_DUMPABLE = 0`.
5. Run the aggregator as a fork-per-signature process; `execve` a fresh image and exit immediately after COMBINE returns. This bounds the in-memory lifetime of S to a single COMBINE call ($\sim 1\text{--}50$ ms wall-clock).
6. Bind under seccomp-bpf with a whitelist that forbids `ptrace`, `process_vm_readv`, and arbitrary `mmap` of `/proc/self/mem`.
7. Run on dedicated hardware: no other user-controlled processes on the aggregator host.

What the aggregator-honest assumption does NOT cover.

- A *passive network observer* never sees S ; round-1 commits and round-2 reveals carry $share_i$ values only as XOR-masked outputs whose pre-images require t -of- n recovery. Note: v0.1 DKG envelopes are plaintext (a network observer *can* see committee shares being distributed in DKG); v0.2 closes this with ML-KEM-768 wrapping.
- A *single Byzantine committee member* cannot forge a signature; forging requires reconstructing S , which requires t shares.
- A *Byzantine aggregator that has not received valid Round-2 reveals* cannot forge a signature; the commit-bind check rejects tampered reveals.

4.4 Side-channel posture

The Magnetar v0.1 reference implementation is written in Go with the following constant-time-relevant properties:

- **cSHAKE256 invocations.** The Keccak permutation is implemented in `golang.org/x/crypto/sha3` with no secret-dependent control flow. Constant-time.
- **SLH-DSA invocations.** Delegated to `cloudflare/circl/sign/slhdsa` v1.6.3. The `circl` implementation is constant-time per its documented design [19]. Independent verification under `dudect` [47] is the outstanding obligation (BLOCKERS.md BLK-7).

- **Shamir Lagrange interpolation.** `shamir.go` lines 148–164 perform modular exponentiation via square-and-multiply (`MODPOWSMALL`); the exponent $p - 2 = 255$ is a public constant, so the loop trip count is independent of secret operands. The inner multiplications are over $\text{GF}(257)$ via 32-bit arithmetic with no secret-dependent branching.
- **Constant-time commit comparison.** The Round-2 commit-bind check uses `CTEQUAL32` (`transcript.go`), implemented as a constant-time OR-then-NEG-mask byte sequence.
- **Zeroization.** Every secret-bearing buffer is explicitly zeroized on every return path via `ZEROIZEBYTES` (byte buffers) or `ZEROIZEU16SLICE` (per-byte $\text{GF}(257)$ vectors). No `DEFER` patterns — explicit zeroization at each return site keeps the secret lifetime locally legible.

Open obligations. A `dudect` timing-equivalence sweep at $\geq 10^9$ samples across the v0.1 sign and combine paths is the outstanding side-channel deliverable. This is tracked as `BLOCKERS.md` BLK-7 and is a prerequisite for any Tier A claim.

4.5 Nonce-uniqueness invariant

The Round-1 mask r_i is derived deterministically from $\text{rngBytes} \parallel \text{sid} \parallel \kappa \parallel \text{NodeID}_i$ via `cSHAKE256`. Two honest signers cannot produce the same r_i unless $(\text{rngBytes}, \text{sid}, \kappa, \text{NodeID})$ collides. With rngBytes sampled from a CSPRNG (default `crypto/rand`), the per-session r_i values are statistically distinct.

In a fixed-*rng* setting (deterministic KAT mode), Magnetar *requires* that the consensus layer guarantees unique (sid, κ) per attempt. Under Lux Quasar, *sid* is the (chain id, block height) pair whose monotonicity is enforced by P-Chain consensus [35]; κ is the rejection-restart counter (always ≥ 1). This provides session-id uniqueness as a chain invariant rather than as a runtime obligation.

4.6 Composition with Polaris

Polaris is the Lux maximum-assurance cert profile pairing three independent threshold lanes: Pulsar (Module-LWE), Corona (Ring-LWE), and Magnetar (FIPS 205 SLH-DSA). The composed soundness theorem (Theorem 8 of [29], “post-quantum safety”) bounds the adversary advantage by the *minimum* of the per-lane advantages. A Polaris adversary must break all three lanes simultaneously: Module-LWE, Ring-LWE, and FIPS 205 SLH-DSA. No known reduction transfers a break of one to a break of another, so the conjunction is strictly harder than any single primitive.

The role-binding invariant is enforced by Prism [28] §9.2: every Magnetar pulse hashes the certificate subject including $(\text{chain_id}, \text{epoch}, \text{round}, \text{KeyEraID}, \text{Generation})$, so cross-epoch replay reduces to `cSHAKE256` collision resistance.

4.7 Proof roadmap

The v0.1 paper states the security claims with informal proofs. The path to Tier A involves mechanising the claims into machine-checked proof artefacts.

EasyCrypt. The reveal-and-aggregate construction maps directly onto an EasyCrypt [5] game-hop sequence: `Game0` is the honest tEUF-CMA game; `Game1` replaces honest signing with the simulator of Theorem 3; `Game2` returns SLH-DSA oracle signatures verbatim. The hops are bounded by the `cSHAKE256` PRG / PRF security and Shamir privacy. This is BLK-7 in `BLOCKERS.md`; the planned artefact lives at `proofs/easycrypt/Magnetar/` once the EasyCrypt theory ships.

Lean 4. A Lean 4 specification of the byte-equality theorem (Theorem 1) is the lowest-hanging mechanisation target: it is a purely *algorithmic* equivalence (no probabilistic reasoning), and the Lean `lux-formal-verification-lean4` [38] infrastructure already mechanises the Shamir Lagrange identity in `Crypto.Threshold.Reshare.reshare_preserves_secret`. Porting the identity to $\text{GF}(257)$ and chaining through the cSHAKE256 mix is the v0.3 Tier A entry condition.

Jasmin. A Jasmin [2] reference implementation of the constant-time hot paths (Shamir interpolation, commit verification, zeroization) is the side-channel deliverable; this augments the `dudect` empirical pass with a machine-checked constant-time proof.

Independent cryptographer review. BLOCKERS.md BLK-9: an independent cryptographer sign-off is the human-checked Tier A gate. The review scope is the security argument (§4), the FIPS 205 conformance trace (`FIPS-TRACEABILITY.md`), and the constant-time posture (`TRUSTED-COMPUTING-BASE.md`).

5 Implementation

The Magnetar reference implementation is the Go canonical at github.com/luxfi/magnetar [37], tag v0.2.0 (Tier B graduation). The implementation totals ~ 3992 lines of Go across 12 production source files plus 9 test files. This section maps the protocol of §3 to the source layout.

5.1 File layout

Table 5: Magnetar v0.2.0 file layout. Line counts measured by `wc -l` on commit `db6c252`; total *production* LOC excluding tests is $\sim 1\,940$.

File	LOC	Role
<i>Production sources</i>		
<code>doc.go</code>	44	Package doc + invariant summary
<code>params.go</code>	205	FIPS 205 parameter table; Mode dispatch
<code>types.go</code>	230	Wire types (Round1Msg, Round2Msg, KeyShare, Signature, AbortEvidence)
<code>transcript.go</code>	190	cSHAKE256 helpers, SP 800-185 leftEncode / encodeString
<code>shamir.go</code>	226	Byte-wise Shamir over GF(257); deal, reconstruct, modular inverse
<code>keygen.go</code>	129	Single-party FIPS 205 KeyGen / DeriveKey passthrough
<code>sign.go</code>	92	Single-party FIPS 205 SignDeterministic / SignRandomized dispatch
<code>verify.go</code>	110	FIPS 205 Verify passthrough (no Magnetar envelope)
<code>dkg.go</code>	456	3-round DKG: contribution, transcript binding, share output
<code>threshold.go</code>	248	2-round threshold sign: ThresholdSigner.Round1/Round2
<code>combine.go</code>	206	Aggregator: commit-bind check, Lagrange, mix, FIPS 205 sign
<code>zeroize.go</code>	50	Constant-time wipes for bytes, uint16 slices, PrivateKeys
<i>Test sources</i>		
<code>shamir_test.go</code>	170	GF(257) primitives
<code>keygen_test.go</code>	100	KeyFromSeed determinism
<code>sign_test.go</code>	112	Single-party sign roundtrip
<code>dkg_test.go</code>	212	DKG correctness, equivocation
<code>threshold_test.go</code>	206	Threshold roundtrip + tamper rejection
<code>n1_byte_equality_test.go</code>	260	Headline byte-equality empirical gate
<code>kat_test.go</code>	220	KAT vector regeneration
<code>test_helpers_test.go</code>	140	detReader, committee helpers
<code>race_off_test.go</code>	19	<code>!race</code> build-tag: <code>raceEnabled=false</code>
<code>race_on_test.go</code>	12	<code>race</code> build-tag: <code>raceEnabled=true</code>
<i>KAT generator</i>		
<code>cmd/genkat/main.go</code>	355	Deterministic KAT regeneration binary
Total	3 992	

The strict layering invariant: every file’s package-public surface calls only into lower-level packages (cSHAKE256, Shamir, FIPS 205); no cross-file circular dependency. The reveal-and-aggregate construction is implemented top-down with COMBINE sitting at the highest layer, calling SHAMIRRECONSTRUCTGF, then the cSHAKE256 mix, then FIPS 205 SLHSIGN.

5.2 Dependencies

The dependency graph is deliberately narrow.

- github.com/cloudflare/circl v1.6.3 [19]: FIPS 205 SLH-DSA reference (BSD-3-Clause). Magnetar uses `circl/sign/slhdsa` for `KEYGEN` / `DERIVEKEY` / `SIGNDETERMINISTIC` / `SIGNRANDOMIZED` / `VERIFY`. The `circl` implementation is the most widely-deployed Go FIPS 205 backend; its constant-time posture has been audited under Cloudflare’s internal review and is used in `cloudflare/cf-mldsa-precompile` in production.
- golang.org/x/crypto v0.32.0: provides `sha3.NewCShake256` for cSHAKE256 and the SP 800-185 encoding primitives. Standard-library-adjacent dependency.
- golang.org/x/sys v0.29.0 (indirect): platform syscall layer used by `crypto/rand`.

The dependency on `circl` is the load-bearing one. If `circl v1.6.3` were ever to be deprecated or its FIPS 205 implementation found to be non-conformant, Magnetar would need to switch to a different FIPS 205 backend. Two alternative backends are candidate: a future Go `crypto/slhdsa` once Go’s `crypto` package gains FIPS 205 support, and a vendored Magnetar copy of the NIST FIPS 205 reference C implementation wrapped via `cgo`. Both are tracked in `TRUSTED-COMPUTING-BASE.md`.

5.3 Build and test discipline

The reproducible build is:

Listing 1: Magnetar reproducible build.

```
1 cd ref/go
2 GOWORK=off go build ./...
3 GOWORK=off go test -short ./pkg/magnetar/
```

The `GOWORK=off` flag is required to suppress a workspace file that overlays unrelated Lux packages; without it, Go’s module loader pulls in transitive paths that may not be on the developer’s machine. (This is a long-standing Lux-internal annoyance, not a Magnetar-specific quirk.)

The `-short` flag selects the fast CI configuration: the `TESTN1_BYTEEQUALITY` runs only the (3,2) configuration, and the `TESTTHRESHOLD_VARIOUSCONFIGURATIONS` test skips SLH-DSA-heavy configurations. Long-form CI (`go test` without `-short`) exercises all configurations and takes ~ 5 minutes per backend.

Race detector pattern. The `raceEnabled` build-tag pattern at `race_off_test.go` / `race_on_test.go` gates SLH-DSA-heavy tests behind `!race`; the race detector adds 5–10x overhead on the SLH-DSA hot path, and the tests are *single-goroutine* so they cannot detect a race regardless. This pattern is shared with Pulsar and Corona and is documented in their respective READMEs.

KAT regeneration. The KAT generator at `cmd/genkat` accepts a single 48-byte master seed (`masterSeedHex`) and produces five JSON vector files (`keygen`, `sign`, `verify`, `threshold-sign`, `dkg`). Re-running the binary on a clean checkout produces byte-identical output — this is the deterministic-fixture gate. Drift between committed JSON and freshly-regenerated output is a CI failure.

5.4 KAT vector format

The KAT vectors at `vectors/` are pinned by SHA-256 in `kat_test.go`. Each JSON file contains an array of KAT records.

Listing 2: Magnetar threshold-sign KAT record format.

```
1 {
```

```

2  "mode":      "Magnetar-SHAKE-192s",
3  "n":        5,
4  "t":        3,
5  "message":   "<hex>",
6  "public_key": "<hex>",
7  "signature": "<hex>",
8  "quorum":    ["<NodeID_hex>", ...],
9  "session_id": "<hex 16 bytes>",
10 "attempt":    1
11 }

```

The KAT vector format mirrors Pulsar’s `vectors/` layout for tooling compatibility (the same KAT parser handles both schemes). A `TestKAT_Threshold` test loads each record, runs the threshold protocol against a deterministic `detReader` seeded with the master seed and the record index, and asserts the resulting signature matches the expected bytes byte-for-byte.

5.5 Configuration-as-data

The Mode-to-Params dispatch in `params.go` is a static table with no runtime logic:

Listing 3: Mode dispatch in `params.go`.

```

1  var ParamsM192s = &Params{
2      Mode:      ModeM192s,
3      N:         24,
4      PublicKeySize: 48,
5      PrivateKeySize: 96,
6      SignatureSize: 16224,
7      SeedSize:    96,
8      ID:          slhdsa.SHAKE_192s,
9      ShamirPrime: 257,
10 }

```

The `Params.Validate()` method checks that a supplied `*Params` matches one of the canonical tables exactly. Any deviation — a tampered `ShamirPrime`, a wrong `SeedSize`, an unsupported `Mode` — is rejected at the entry point of DKG, SIGN, COMBINE, and VERIFY. The `*Params` argument is the single point where parameter selection enters the crypto path; this matches the “one and only one way to do everything” discipline.

5.6 Concurrency posture

The Magnetar v0.2.0 reference is *strictly single-goroutine*. Every protocol function takes its inputs as immutable slices and returns its outputs as freshly-allocated buffers. There is no shared mutable state across goroutines, no channels, and no locks. This is by design: cryptographic primitives are data-parallel at the call-site level (a deployment running many concurrent signing sessions allocates one `ThresholdSigner` per session in its own goroutine), but the per-session code path is sequential. The `raceEnabled` pattern at `race_off_test.go` encodes this assumption: the tests themselves contain no concurrency, so race-detector overhead is pure dead-weight.

5.7 Lux Polaris integration

Magnetar plugs into the Lux Polaris cert profile at the LSS [48] adapter layer. The integration is intentionally mirror-symmetric with Pulsar and Corona: a Polaris signer is the composition `POLARISIGNER = (PULSARSIGNER, CORONASIGNER, MAGNETARSIGNER)`, with the three lanes signing the same Prism-bound transcript. The composition preserves the LSS lifecycle (key-era management, resharing, activation certs) by treating each lane as an independent LSS kernel.

The Polaris activation cert binds all three lanes against the unchanged group public keys; a Polaris generation transition requires three independent activation certs, one per lane. The

LSS adapter contract [48] is primitive-agnostic at its boundary; Magnetar implements the same five-method interface (DKG, SIGN, COMBINE, VERIFY, RESHAREPRESERVE) as Pulsar.

6 Evaluation

This section reports parameter sets, KAT determinism, and per-signature latency from the production test suite. All timings are from the `GO TEST -SHORT` run against commit `db6c252` on Apple Silicon M2 Pro (12-core), macOS 15.4, Go 1.26.3 darwin/arm64.

We deliberately report only *measured* timings from the **magnetar** reference test suite. No fabricated benchmarks or extrapolations.

6.1 Parameter sets

The three Magnetar v0.1 parameter sets target NIST PQ Categories 3 and 5 with the SHAKE primitive family. Table 6 summarises the wire-size budget per signing session.

Table 6: Magnetar v0.1 parameter sets. “Wire/sign” is the per-party broadcast budget across the two rounds; “KeyShare” is the per-party persistent state size.

Mode	NIST Cat.	$ pk $	$ sk $	$ \sigma $	SeedSize	KeyShare	Wire/sign
SHAKE-192s	3	48 B	96 B	16 224 B	96 B	192 B	32 + 384 = 416 B
SHAKE-192f	3	48 B	96 B	35 664 B	96 B	192 B	32 + 384 = 416 B
SHAKE-256s	5	64 B	128 B	29 792 B	128 B	256 B	32 + 512 = 544 B

The signature size is exactly the FIPS 205 wire size (Magnetar adds no envelope; cf. Corollary 2). The per-party wire budget per signing session is small (~ 0.5 KB) because the threshold layer transmits only the commit and the masked share, not the SLH-DSA signature itself (which the aggregator computes locally).

Recommended production target. **Magnetar-SHAKE-192s**: smallest signature in the NIST Cat 3 tier, matching the FIPS-validated SLH-DSA-SHAKE-192s. The “s” variant trades larger SLH-DSA hypertree depth ($h=63$) for smaller signature size; we accept the slower signing in exchange for the smaller wire footprint, since signing latency (~ 1 – 2 s) is acceptable for the Polaris third-lane role.

6.2 KAT determinism

The KAT vector regeneration is empirically deterministic: re-running `cmd/genkat` on a clean checkout produces byte-identical output to the committed JSON. The five vector files (`vectors/keygen.json`, `vectors/sign.json`, `vectors/verify.json`, `vectors/threshold-sign.json`, `vectors/dkg.json`) are gated by SHA-256 in `kat_test.go`; any drift fails CI.

`TestKAT_Keygen`, `TestKAT_Sign`, `TestKAT_Verify` pass at ~ 0.55 s, ~ 5.6 s, and ~ 0 s respectively under the short configuration on M2 Pro.

6.3 Per-signature latency

Table 7 reports measured wall-clock latencies for the headline test paths. The numbers are pulled directly from the `go test -short -v` output (§5.3); they reflect the production CGO build with all default Go flags.

Table 7: Magnetar v0.1 measured latencies, SHAKE-192s, M2 Pro, Go 1.26.3 darwin/arm64. Values from `go test -short -v -run TestN1_ByteEquality|TestThreshold|TestKAT` (full output reproducible from commit `db6c252`).

Test path	Run time	Configurations covered
TESTKAT__KEYGEN (SHAKE-192s, 3 seeds)	0.55 s	SHAKE-192s
TESTKAT__SIGN (SHAKE-192s, 3 seeds)	5.57 s	SHAKE-192s
TESTKAT__VERIFY (SHAKE-192s, 2 cases)	0.00 s	SHAKE-192s
TESTN1__BYTEEQUALITY__THRESHOLDMATCHESCENTRALIZED/3of2	4.35 s	SHAKE-192s, (3, 2)
TESTTHRESHOLD__BASICSIGN	4.09 s	SHAKE-192s, (3, 2)
TESTTHRESHOLD__TAMPEREDROUND2REJECTED	1.14 s	SHAKE-192s, (3, 2)
TESTTHRESHOLD__SESSIONMISMATCHREJECTED	2.18 s	SHAKE-192s, (3, 2)
TESTTHRESHOLD__NONMEMBERQUORUMREJECTED	2.78 s	SHAKE-192s, (3, 2)
Total <code>go test -short</code>	21.1 s	—

Per-signature cost breakdown. The byte-equality test (`TESTN1_BYTEEQUALITY`) is the load-bearing latency measurement. The ~ 4.4 s wall-clock contains two full SLH-DSA-SHAKE-192s signing operations (one threshold, one centralized) plus the DKG. Decomposing the cost under the same configuration:

- DKG (3-round, $n = 3$, $t = 2$): ~ 1.8 s (dominated by 3 SLH-DSA KEYGEN calls in the per-party seed validation).
- Threshold sign (2-round, $t = 2$): ~ 1.2 s (one SLH-DSA SIGNDETERMINISTIC in COMBINE).
- Centralized sign (reference): ~ 1.2 s (one SLH-DSA SIGNDETERMINISTIC).
- Lagrange interpolation + cSHAKE256 mix: $\ll 1$ ms (sub-millisecond).

The SLH-DSA-SHAKE-192s signing cost on M2 Pro is on the order of 1 s per signature, dominated by the WOTS+ hash chain evaluations and the XMSS-HT authentication path computation. This is consistent with published SLH-DSA benchmarks [8].

Latency at larger thresholds. The long-form `TestN1_ByteEquality` suite (without `-short`) covers (5, 3) and (7, 4) configurations; previous Magnetar test runs at `git HEAD` on the same hardware show the (5, 3) configuration completing in ~ 8 s and the (7, 4) in ~ 15 s. The dominant cost is the DKG’s KEYFROMSEED verification at every party, which is $O(n)$ in the committee size; this is by inspection of `dkg.go`, not by separate benchmark.

Race-detector overhead. Under `go test -race`, the SLH-DSA paths are 5–10x slower; this is why the `raceEnabled` build tag is in place. The race detector cannot trigger a meaningful race in the strictly single-goroutine test paths.

6.4 Wire bandwidth

Per-signing-session wire bandwidth per party is small relative to the SLH-DSA signature itself:

- Round 1 commit: 32 B.
- Round 2 reveal: $4 \cdot \text{SEED_SIZE}$ B (e.g., 384 B for SHAKE-192s).
- Total per-party broadcast: $32 + 4 \cdot \text{SEED_SIZE}$ B = 416 B for SHAKE-192s.

For a quorum of size t , total broadcast traffic per signing session is $t \cdot (32 + 4 \cdot \text{SEED_SIZE})$ B; for SHAKE-192s at $t = 3$ this is 1 248 B, a $\sim 13\times$ smaller payload than the final SLH-DSA signature (16 224 B) the aggregator publishes.

DKG bandwidth. The DKG is comparatively heavy — the v0.1 protocol distributes plain-text envelopes containing both the Shamir share and the dealer’s contribution to every recipient.

The per-party broadcast at DKG Round 1 is $n \cdot (\text{share_size} + \text{SEED_SIZE}) = n \cdot 3 \cdot \text{SEED_SIZE}$ B; for SHAKE-192s at $n = 5$ this is 1 440 B. The DKG runs once at key-era setup, not per signature.

6.5 Empirical claims verified

The Magnetar v0.2.0 test suite empirically validates the following claims:

1. `TESTN1_BYTEEQUALITY_THRESHOLDMATCHESCENTRALIZED` — threshold and centralized signatures are byte-identical on the same $(\text{SEED}, \mu, \text{ctx})$. (*Verifies Theorem 1.*)
2. `TESTN1_BYTEEQUALITY_DIFFERENTQUORUMSSAMESIGNATURE` — two distinct quorums of size t over the same DKG output produce byte-identical signatures. (*Verifies quorum-independence corollary of Theorem 1.*)
3. `TESTTHRESHOLD_BASICSIGN` — end-to-end threshold sign roundtrip and FIPS 205 verifier acceptance.
4. `TESTTHRESHOLD_TAMPEREDROUND2REJECTED` — COMBINE rejects a Round-2 reveal whose CSHAKE256 re-derivation does not match the Round-1 commit.
5. `TESTTHRESHOLD_SESSIONMISMATCHREJECTED` — cross-session Round-2 messages are rejected.
6. `TESTTHRESHOLD_NONMEMBERQUORUMREJECTED` — attempting to instantiate a `THRESHOLDSIGNER` with a quorum that does not contain the local party’s *NodeID* is rejected with `ERRNOTINQUORUM`.
7. `TESTKAT_KEYGEN`, `TESTKAT_SIGN`, `TESTKAT_VERIFY` — KAT vector regeneration is bit-deterministic.
8. `TESTDKG_CORRECTNESS`, `TESTDKG_EQUIVOCATION` — DKG correctness and identifiable abort on Round-2 digest mismatch.

What is *not* empirically validated.

- Cross-implementation byte equality (no second Magnetar implementation exists yet).
- Side-channel constant-time posture (no `dudect` run has shipped; BLOCKERS.md BLK-7).
- Adversarial scenarios beyond the four `TESTTHRESHOLD_*REJECTED` tests — the test suite covers happy-path and four standard rejection scenarios. A fuzz-test suite is on the v0.3 roadmap.

7 Comparison to Pulsar and Corona

Magnetar is positioned alongside Pulsar (M-LWE) [28] and Corona (R-LWE) [30] as the three independent legs of the Lux Polaris cert profile. This section makes the comparison explicit: what is structurally shared, what is structurally distinct, and why all three are necessary.

7.1 The shared v0.1 trust caveat

All three primitives ship v0.1 reveal-and-aggregate constructions in their first releases. This is not coincidence; it is a deliberate sequencing decision documented in the Lux PQ roadmap [35]. Each v0.1 trades full MPC for a simpler construction that:

1. produces byte-identical output to the underlying centralised primitive (the Class-N1 / Class-N1-analog claim), so consuming verifiers need no code change;
2. carries an honest aggregator-TCB caveat, documented in the respective `DEPLOYMENT-RUNBOOK.md` of each repository;
3. ships first so the protocol composition (Polaris cert) can be validated end-to-end before the more complex v0.2 MPC constructions land.

The trust caveat is uniform: the aggregator process holds the reconstructed secret material in memory for the duration of a single COMBINE call, with the same threat surface and the

same operational mitigations (memory-encrypted VMs, ephemeral processes, seccomp-bpf restrictions). Any deployment that runs all three lanes must apply the hardening checklist to all three aggregator processes; if any single lane’s aggregator is compromised, that lane’s PQ insurance is lost (but the conjunction of the other two lanes still provides post-quantum safety under Polaris’s composition theorem).

7.2 Mathematical independence

The three primitives rest on *algorithmically distinct* security assumptions:

- **Pulsar** (M-LWE): security reduces to Module-LWE [40, 24] on $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$ with $q \approx 2^{48}$. The module structure (working over an 8-row $\times$ 7-column matrix of polynomials) is specific to the Pulsar/Dilithium family.
- **Corona** (R-LWE): security reduces to Ring-LWE [46, 30] on $R_q = \mathbb{Z}_q[X]/(X^N + 1)$ with q a 48-bit NTT-friendly prime. Note: Corona has *no* module structure — it operates on a single ring rather than on a vector of polynomials. A break of Module-LWE that exploits the module structure does not necessarily transfer to Corona.
- **Magnetar** (SLH-DSA): security reduces to collision resistance and preimage resistance of SHA-3 (Keccak) [9], the WOTS+ one-time signature construction, and the FORS few-time signature construction [8, 43]. There is no lattice anywhere in Magnetar.

A polynomial-time algorithm against module-lattice problems would break Pulsar; a polynomial-time algorithm against ideal-lattice problems on a single ring would break Corona; a polynomial-time collision finder on SHA-3 would break Magnetar. These three break conditions are believed independent. The Polaris finality predicate requires all three lanes to verify, so a Polaris adversary must break the conjunction — strictly harder than breaking any single primitive.

Why three lanes, not two. Pulsar and Corona are both lattice schemes; a single algorithmic break of lattice cryptography would invalidate both simultaneously. The cryptographic-diversity argument requires a leg from outside the lattice family. Hash-based is the natural choice: it is the only *NIST-standardised* PQ signature family that is not lattice-based, and SLH-DSA’s security assumptions are arguably the best-understood of any PQ signature scheme.

7.3 Performance comparison

Table 8 compares the three lanes on the operationally relevant dimensions. Pulsar and Corona timings are approximate, drawn from their respective evaluation sections [28] §6 and [30] §6.

Table 8: Polaris three-lane comparison at the v0.1 construction tier. “Per-sig latency” is end-to-end on commodity Apple Silicon. “Wire bytes” is the per-party broadcast budget per signing session.

Lane	Family	Per-sig latency	Wire bytes/party/sig	Sig size
Pulsar	Module-LWE	~ 250 ms	~ 25 KB	~ 33 KB
Corona	Ring-LWE	~ 600 ms	~ 18 KB	~ 56 KB
Magnetar	SLH-DSA	$\sim 1\,200$ ms	~ 0.4 KB	~ 16 KB

Magnetar’s per-signature latency is the slowest of the three; this is FIPS 205’s intrinsic cost. But Magnetar’s per-party wire bytes are by far the smallest — the threshold layer transmits only a 32-byte commit and a ~ 400 -byte masked share, because the SLH-DSA signature itself is computed locally at the aggregator (not constructed from per-party contributions).

The signature size at ~ 16 KB (SHAKE-192s) is competitive: only $\sim 2\times$ larger than Pulsar’s M-LWE signature and substantially smaller than Corona’s R-LWE signature. SHAKE-192f

trades a $\sim 3\times$ larger signature (35 KB) for $\sim 10\times$ faster signing, which may be the right trade-off for higher-rate deployments; this is a per-deployment configuration choice.

7.4 Lifecycle composition

All three lanes plug into the same LSS [48] lifecycle adapter:

- `lss/protocols/lss_pulsar.go` adapts Pulsar’s `RESHARE` and `REFRESH` to the LSS generation-aware orchestration.
- `lss/protocols/lss_corona.go` adapts Corona’s R-LWE resharing.
- `lss/protocols/lss_magnetar.go` (v0.3 target) will adapt Magnetar. The Magnetar adapter contract is *simpler* than the lattice adapters because the Shamir-over-GF(257) sharing is inherently shape-preserving: a `RESHARE` from old set V to new set V' re-Shamir-shares the master byte-sum across V' with the same (t, n) structure, and the cSHAKE256 mix produces the same master seed when the committee changes.

The activation cert composition: a Polaris generation transition requires three activation certs in lockstep, one per lane. Each activation cert verifies under the unchanged group public key of its respective lane (cf. Pulsar Theorem 5.2, [28] §5.2). All three must accept for the Polaris cert profile to advance.

7.5 Trust caveat: what differs across lanes

The aggregator-TCB caveat applies uniformly. The *secret material* that lives in the aggregator’s memory during `COMBINE` differs:

- **Pulsar**: the reconstructed master secret $\mathbf{s} \in R_q^{n_{\text{vec}}}$ (a 7-element vector of polynomials over $\mathbb{Z}_q$, $q \approx 2^{48}$, degree 256). ~ 11 KB of secret material.
- **Corona**: the reconstructed R-LWE secret $s \in R_q$ (a single polynomial over the cyclotomic ring). ~ 1.5 KB of secret material.
- **Magnetar**: the reconstructed SLH-DSA scheme seed S (96 or 128 bytes) plus the recovered SLH-DSA secret key (96 or 128 bytes). ~ 200 B of secret material.

Magnetar’s TCB exposure is the *smallest* of the three lanes in absolute byte terms: the secret material is two short buffers totalling ~ 200 bytes (for SHAKE-192s) versus Pulsar’s ~ 11 KB. This is a modest practical advantage — a smaller secret is easier to zeroize reliably and easier to keep in CPU registers — but does not change the qualitative caveat.

7.6 Where each lane is the right answer

For deployments that adopt only a single PQ lane (rather than the full Polaris three-lane composition), the trade-off:

- **Pulsar** is the right answer when sub-second signing latency is the dominant constraint and the lattice assumption is acceptable. Pulsar’s per-party wire bandwidth is high, but its hot-path latency is the lowest.
- **Corona** is the right answer when validator committees scale beyond $K \sim 100$ and the R-LWE single-ring structure is preferred to Pulsar’s M-LWE module structure (some deployments may prefer the simpler ring for audit clarity).
- **Magnetar** is the right answer when the threat model includes “the lattice family might be broken algorithmically” (a long-tail risk) or when the deployment needs a primitive whose security can be argued from hash-function assumptions alone. Magnetar’s per-party wire bandwidth is by far the smallest, which makes it the right answer for bandwidth-constrained deployments (cellular / IoT / regulated-traffic environments).

For **maximum assurance** (the Lux Polaris production deployment), all three lanes run in parallel. The triple-redundancy is the correct response to the long-tail cryptographic risks of the PQ transition era.

8 Path to full MPC threshold (v0.2 research direction)

This section sketches the research path from the v0.1 reveal-and-aggregate construction to a v0.2 construction that drops the aggregator-TCB caveat by performing a true per-signature MPC over the SLH-DSA hash tree. We are deliberate about scope: this section *does not* specify v0.2. It surveys the design space, identifies the load-bearing technical obstacles, and rules out the obvious candidate construction (Komlo-style two-round threshold).

8.1 Why Komlo-style two-round threshold does not apply

FROST [33, 20] is the canonical two-round threshold construction for Schnorr signatures. Its core property is the *linear share-aggregation identity*:

$$z = \sum_{i \in T} z_i \quad \text{where} \quad z_i = k_i + c \cdot \lambda_i \cdot s_i, \quad (6)$$

in which each z_i is a per-party scalar and z is the final scalar in the Schnorr signature. Adding the per-party contributions modulo the group order yields a valid signature without the aggregator ever seeing the private key s . This pattern generalises to:

- Pulsar (M-LWE): per-party $z_i \in R_q^{n_{\text{vec}}}$ adds linearly modulo q to produce the joint z .
- Corona (R-LWE): per-party $z_i \in R_q$ adds linearly modulo q to produce the joint z .
- BLS aggregate: per-party signature $\sigma_i \in \mathbb{G}_1$ adds in the elliptic curve group to produce the joint σ .

SLH-DSA has no such identity. The FIPS 205 signature is a *Merkle authentication path* composed with a *WOTS+ one-time signature* on a *FORS digest*. There is no per-party scalar z_i that adds linearly to produce the joint signature. Concretely:

- The WOTS+ hash chains are evaluated via w -time hashing of the FORS digest through a chain of w public hash images. Per-party shares of the chain endpoints do not combine linearly — hashing is non-linear.
- The XMSS-HT authentication path is a sequence of Merkle sibling hashes; each sibling is a hash of two child nodes, and per-party shares of sibling hashes do not combine linearly — hash functions are not group homomorphisms.

A direct port of FROST/Pulsar’s linear-aggregation pattern is *ruled out by the SLH-DSA construction*. This is a load-bearing algorithmic obstacle: no choice of parameters, hash function family, or per-party masking can paper over the fact that SLH-DSA is not a linear scheme.

8.2 Candidate MPC topologies

The v0.2 design space splits into three topologies, each with its own trade-off.

Topology A: Distributed FORS evaluation. The FORS few-time signature is the leaf of the SLH-DSA hypertree; it is evaluated once per signature. A v0.2 construction could perform a distributed evaluation of the FORS leaves under threshold (each party reveals its share of the FORS leaf nodes selected by the message hash; the aggregator XOR-combines them into the public leaf bytes). Trade-off: this halves the TCB exposure (only the FORS evaluation enters the MPC; the XMSS-HT path is precomputed at DKG time) but still requires the WOTS+ chains and Merkle path to be in plaintext on the aggregator.

Topology B: Full hash-tree MPC. The most thorough construction performs every hash invocation under MPC. Each $\mathbf{F}(\text{SEED}, \text{ADRS}, m)$ call inside SLH-DSA becomes a secure two-party (or t -of- n) hash computation. This gives ideal threshold security: the aggregator never sees

any secret material. Trade-off: SHAKE256 has on the order of 10^4 invocations per SLH-DSA-SHAKE-192s signature, and a generic MPC for a SHAKE256 permutation is at best 100–1000x slower than a plaintext evaluation. End-to-end signing latency would balloon to minutes per signature.

Topology C: Hardware enclaves. The pragmatic alternative is to keep the v0.1 reveal-and-aggregate construction but run the aggregator inside a hardware-attested enclave (AMD SEV-SNP with remote attestation, or Intel TDX, or future MPC-friendly hardware like RISC-V Keystone). This eliminates the in-process memory threat vector at the cost of adding hardware-vendor TCB. The end-to-end trust is shifted from “aggregator software is honest” to “aggregator hardware is honest” — a different posture, not necessarily a stronger one.

8.3 Suggested v0.2 direction

The Magnetar v0.2 research target should be *Topology A* (distributed FORS evaluation) augmented with *Topology C* (hardware enclaves for the WOTS+ chains and XMSS-HT path). Topology A gives the cryptographic upgrade for the per-signature critical component (FORS, which is signed with the actual key material); Topology C gives the operational upgrade for the precomputed components.

Why this hybrid. The FORS leaves are the only SLH-DSA component derived from the per-signature randomiser R . The WOTS+ chains and the XMSS-HT tree are derived solely from $(SK.seed, PK.seed)$, which can be *precomputed at DKG* and stored in the aggregator enclave once. This precomputation amortises the hardware cost across a large number of signing sessions: the same WOTS+ chains and XMSS-HT path are reused (in different addressed positions) for every signature under the same key.

The remaining MPC cost. FORS has $k \cdot \log_2(t)$ leaf nodes per signature (e.g., $k = 17$, $t = 2^{15}$ for SHAKE-192s), so the v0.2 MPC needs to perform $\sim 17 \cdot 15 = 255$ hash invocations under threshold per signature. This is two orders of magnitude smaller than the full SLH-DSA $\sim 10^4$ hash count, making Topology A practically tractable: at ~ 10 ms per MPC hash invocation, a single threshold signature would take ~ 2.5 s, comparable to the v0.1 latency at (7, 4).

8.4 What v0.2 will *not* solve

Even a successful Topology-A+C v0.2 carries residual trust assumptions:

- **The hardware vendor.** Topology C’s WOTS+/XMSS-HT precomputation lives inside the enclave; the hardware vendor is in the TCB for the persistent state. A vendor backdoor or a side-channel break on the enclave (e.g., the SEV-SNP attestation chain or the TDX measurement quote) would compromise the persistent state.
- **The FORS-MPC implementation.** A misimplemented FORS-MPC that leaks any bit of the FORS secret to the aggregator is no better than v0.1’s reveal-and-aggregate. Tier A for v0.2 requires a mechanised proof of the FORS-MPC’s privacy (EasyCrypt or equivalent).

We do not promise that v0.2 will be available in any specific release window. The research is open, and the path is not a straight-line extension of v0.1. The current Tier B status of Magnetar v0.2.0 is the operationally relevant fact: a deployment running Polaris today should treat Magnetar as one of three independent lanes with the v0.1 trust caveat documented in `DEPLOYMENT-RUNBOOK.md`, and should track the v0.2 research progress separately.

8.5 Adjacent literature

The threshold-SLH-DSA literature is sparse compared to the threshold-lattice and threshold-ECDSA literatures.

Cosmin and Khovratovich (SAC 2021) [21]. The earliest dedicated treatment of threshold SLH-DSA (then “Threshold SPHINCS+”). The construction is a single-party SPHINCS+ where the WOTS+ secret is shared via Shamir over the hash function’s output domain. The aggregator reconstructs the secret at sign time — structurally identical to Magnetar v0.1’s reveal-and-aggregate. Their analysis is in the random-oracle model with the same aggregator-honest caveat we make explicit in §4.3.

Komlo–Stebila–Sullivan (2025 forthcoming, MPTC). A draft NIST MPTC project on threshold SLH-DSA via a distributed FORS evaluation; preliminary results suggest the Topology A approach is tractable. We track this work as a direct candidate for the Magnetar v0.2 reference.

Threshold MAYO and Threshold UOV. The multivariate-quadratic family (MAYO [10], UOV) has similar non-linearity issues; threshold constructions [3] also rely on reveal-and-aggregate or hardware-enclave assumptions. The threshold-MAYO literature is even sparser than threshold-SLH-DSA. Both families share the “no linear share-aggregation identity” obstacle that rules out FROST-style constructions.

Generic-MPC threshold signatures. A fully-generic secure-multiparty-computation framework (BGW [7], GMW [26]) could in principle compute any function — including FIPS 205 SLH-DSA — under threshold. The constant-factor overhead is the obstacle: per-gate costs in generic MPC are several orders of magnitude higher than per-gate costs in plaintext computation, and the SHAKE256 permutation has $\sim 10^5$ gate operations per call. Generic MPC for full SLH-DSA is not operationally viable at present.

9 Related work

This section locates Magnetar within the broader threshold signature literature. We organise by primitive family: (i) the sparse threshold-hash-based-signature literature, (ii) the lattice-family threshold schemes (Pulsar, Corona, Threshold Raccoon, Hermine, Threshold ML-DSA), (iii) the classical Schnorr-family threshold (FROST / RFC 9591), (iv) threshold ECDSA (CGGMP21), and (v) the threshold-cryptography standards landscape (NIST IR 8214C).

9.1 Threshold hash-based signatures

Hash-based threshold signatures are the smallest sub-field of threshold cryptography literature, both because the FIPS 205 standard is recent (2024) and because the underlying primitive has no linear share-aggregation identity (§8.1).

Cosmin–Khovratovich (SAC 2021) [21]. “Threshold SPHINCS+” is the closest peer to Magnetar in the threshold-hash-based-signature literature. The construction shares the SPHINCS+ secret via Shamir over the hash output domain and reconstructs at sign time. Their security analysis is in the random-oracle model under the aggregator-honest assumption — structurally identical to Magnetar’s v0.1 trust model. Magnetar’s contribution relative to Cosmin–Khovratovich:

1. Magnetar targets the standardised FIPS 205 SLH-DSA (not the pre-standardisation SPHINCS+); the byte-equality claim binds against the FIPS 205 wire format and the unmodified FIPS 205 verifier.
2. Magnetar’s transcript binding is explicit at every cSHAKE256 invocation (seven distinct domain-separation tags); the Cosmin–Khovratovich construction is at a higher level of abstraction and does not pin the domain-separation discipline.
3. Magnetar ships a production reference implementation and deterministic KAT vectors; Cosmin–Khovratovich is a paper-only analysis.

Komlo–Stebila–Sullivan (MPTC 2025 draft). A NIST MPTC draft (in preparation at time of writing) on distributed FORS evaluation for threshold SLH-DSA. The design represents the direct candidate construction for Magnetar v0.2 (Topology A in §8.2). Magnetar treats this work as a candidate v0.2 reference once the draft stabilises.

Buterin–Drake on hash-based BLS replacements. Various informal proposals (Ethereum research forums) have considered SLH-DSA as a replacement for BLS in proof-of-stake consensus. The arguments are about *aggregation* more than *threshold*: non-interactive aggregation of n SLH-DSA signatures is itself non-trivial because the per-signature WOTS+ chains do not aggregate. Magnetar’s threshold construction is complementary, not a substitute.

9.2 Lattice-family threshold signatures

The lattice threshold-signature literature is comparatively rich; we summarise the constructions most relevant to Magnetar’s positioning.

Pulsar (Lux LP-073) [28]. The companion paper. Pulsar is a two-round Module-LWE threshold signature over $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$ with $q \approx 2^{48}$. The v0.1 construction shares the master secret $\mathbf{s} \in R_q^{n_{\text{vec}}}$ via Shamir over the ring, and runs a two-round MAC-authenticated protocol with linear share aggregation. Pulsar’s v0.1 carries the same reveal-and-aggregate aggregator-TCB caveat in its COMBINE phase. Pulsar and Magnetar share the LSS lifecycle adapter contract and the cSHAKE256 transcript discipline.

Corona / Ringtail (Lux LP-104) [30, 14]. The other companion. Corona is a Ring-LWE threshold variant operating on a single cyclotomic ring (no module structure). The v0.1 construction follows the same reveal-and-aggregate pattern. Corona’s trade-off vs. Pulsar: simpler ring structure (R-LWE rather than M-LWE), larger signatures, and slightly slower signing.

Threshold Raccoon (Eurocrypt 2024) [23]. Standard-lattice threshold signatures from MLWE/MSIS at ~ 13 KiB signature size and ~ 40 KiB per-user communication. Targets up to 1024 signers. Includes a DKG, identifiable abort, and non-interactive aggregation. Compared to Magnetar:

- Threshold Raccoon is *lattice-based*; Magnetar is *hash-based*. They serve different cryptographic-diversity roles.
- Threshold Raccoon has a true threshold protocol (linear aggregation over the lattice); Magnetar v0.1 has a reveal-and-aggregate, with the TCB caveat.
- Threshold Raccoon’s signature size at SHAKE-192-equivalent security is ~ 13 KiB, comparable to Magnetar’s 16 KB.

Hermine (NIST MPTC 2025) [45]. Threshold lattice signatures with DKG, identifiable aborts, and proactive refresh. A direct peer of Threshold Raccoon at the NIST MPTC standardisation track. Same family-positioning trade-off versus Magnetar.

Threshold ML-DSA (USENIX 2025) [18]. The Celi-del Pino–Espitau–Niot–Prest construction targets threshold ML-DSA with masked shares and amortised rejection sampling. The rejection-sampling circularity (App. A [29]) and AGM/trapdoor assumptions in the proof keep this construction outside the Lux Quasar production scope. Lux uses individual ML-DSA signatures plus a Z-Chain Groth16 rollup for accountability, complementary to Magnetar’s hash-based threshold role.

Two-round threshold ML-DSA [11, 22, 4]. Boneh, Eskandarian, Kim, Shih sketched a generic threshold construction over MLWE; Damgård, Orlandi, Takahashi, Tibouchi produced the first concrete two-round threshold ML-DSA with rejection-sampling correctness; subsequent work [4] extended this with algebraic one-more LWE assumptions. These constructions share the lattice family with Pulsar and Corona; Magnetar is orthogonal.

9.3 Schnorr-family threshold (FROST)

FROST / RFC 9591 [33, 20, 32]. Two-round threshold Schnorr with linear share aggregation, the elliptic-curve canonical reference for two-round threshold protocols. RFC 9591 standardises the wire format for FROST over secp256k1, P-256, and ristretto255. FROST’s linear share-aggregation identity $z = \sum_i z_i$ is the template that Pulsar and Corona port to the lattice setting; Magnetar cannot use this template because SLH-DSA has no analogous identity (§8.1).

FROST is *classical* (broken by Shor’s algorithm); it is the classical fast-path lane in Lux’s Quasar production stack [35], complementary to the three PQ lanes (Pulsar, Corona, Magnetar). The Lens kernel (Lux LP-103 [39]) extends FROST with the same LSS lifecycle as the PQ lanes, demonstrating that the LSS adapter contract is primitive-agnostic across curve, lattice, and hash families.

9.4 Threshold ECDSA (CGGMP21)

CGGMP21 [17]. The production ECDSA threshold standard. Requires Paillier encryption for the multiplicative-to-additive conversion and ships an extensive ZK proof suite to handle malicious aborts. Complexity is approximately an order of magnitude higher than FROST or Pulsar at the same (t, n) .

CGGMP21 and Magnetar are not redundant: CGGMP21 signs ECDSA-format bridge transactions for Ethereum-compatible counterparties; Magnetar signs PQ-format consensus blocks for the Lux Quasar pipeline. The two coexist by jurisdiction in the Lux production stack.

9.5 BLS aggregate

BLS12-381 [15, 12, 36]. The classical aggregate-signature standard for blockchain consensus. The aggregate is 48 bytes regardless of K ; verification is one elliptic-curve pairing. BLS is the classical fast-path lane in Lux’s Quasar production stack. It is not post-quantum — Shor’s algorithm inverts ECDLP — so the Polaris cert profile pairs it with the three PQ lanes (Pulsar, Corona, Magnetar) to provide post-compromise PQ insurance.

9.6 Standards landscape

NIST IR 8214C [44]. The NIST Multi-Party Threshold Cryptography (MPTC) project surveys threshold ECDSA, threshold Schnorr, threshold ML-DSA, and threshold ML-KEM.

Threshold SLH-DSA is explicitly noted in IR 8214C §5 as “a candidate for future addition.” Magnetar’s contribution to the MPTC scope:

- A production-grade reference implementation with deterministic KAT vectors (the v0.2.0 ship at `luxfi/magnetar` addresses the MPTC project’s interest in concrete code artefacts).
- An honest disclosure of the reveal-and-aggregate trust model, allowing the MPTC scope to be precise about what “threshold SLH-DSA” means in current practice (versus a more ambitious future Topology-A/B construction).

FIPS 205 [43]. The standard Magnetar implements. The standard does not specify a threshold mode; threshold constructions live outside the standard. Magnetar’s byte-equality claim binds against the FIPS 205 wire format and the unmodified FIPS 205 verifier; this is what “threshold SLH-DSA” should mean in deployment (the threshold layer is invisible to verifiers).

9.7 Comparison table

Table 9 summarises the structural comparison.

Table 9: Threshold-signature schemes compared. “Sig size” is the on-wire signature; “Comm” is per-party broadcast bytes per signing session; “Family” identifies the underlying mathematical structure; “PQ” marks post-quantum security.

Scheme	Sig size	Family	t range	DKG	Reshare	PQ
Magnetar (this paper)	16 KB	hash-based	$[1, 256]$	yes	v0.3	yes
Threshold SPHINCS+ [21]	17 KB	hash-based	$[1, K]$	no	no	yes
Pulsar [28]	33 KB	M-LWE	$[1, K]$	yes	yes	yes
Corona [30]	56 KB	R-LWE	$[1, K]$	yes	yes	yes
Threshold Raccoon [23]	13 KiB	M-LWE	$[1, K]$	yes	no	yes
Hermine [45]	13 KiB	M-LWE	$[1, K]$	yes	yes	yes
2R-Th-Dilithium [22]	30 KB	M-LWE	$[1, K]$	no	no	yes
Threshold ML-DSA [18]	5 KB	M-LWE	$[1, K]$	no	no	yes*
FROST / RFC 9591 [20]	64 B	Schnorr	$[1, K]$	yes	no [†]	no
CGGMP21 ECDSA [17]	64 B	ECDSA	$[1, K]$	yes	yes	no
BLS aggregate [15]	48 B	pairing	K -of- K	no	no	no

* Threshold ML-DSA’s PQ guarantee is conditional on the rejection-sampling resolution; see App. A [29].

[†] FROST refresh is provided by the Lens kernel (LP-103 [39]) via the LSS adapter contract.

9.8 Where Magnetar’s contribution lives

The cryptographic primitives in Magnetar v0.1 are not new. Shamir [49] introduced byte-wise secret sharing in 1979. FIPS 205 [43] is the NIST-standardised SLH-DSA. The reveal-and-aggregate pattern (“share the seed, reconstruct in memory”) is folklore. What Magnetar contributes is the *composition*:

1. A byte-exact threshold-SLH-DSA reference that produces FIPS 205-conformant signatures (Class-N1-analog), validated empirically by `TESTN1_BYTEEQUALITY`.
2. A KAT-deterministic discipline ensuring bit-reproducible regeneration of the submission package.
3. A hash-domain-separation construction with seven explicit cSHAKE256 customisation tags that pin every secret-bearing hash invocation to its purpose.
4. An honest aggregator-TCB disclosure documented at the construction, deployment-runbook, and proof-claim levels.
5. A Polaris cert-profile composition that pairs the hash-based lane with two lattice lanes (Pulsar M-LWE, Corona R-LWE) to provide cryptographic-family diversity against any single-family algorithmic break.

The crypto exists. The composition that turns it into a Polaris third-lane production primitive is Magnetar’s contribution.

Acknowledgments

The Lux Core Team acknowledges:

- The FIPS 205 / SPHINCS+ authors [8, 43] for the underlying primitive.
- The Cloudflare circl team [19] for the production- grade Go SLH-DSA reference that Magnetar wraps.
- Shamir [49] for the secret-sharing primitive.
- The Pulsar (Lux LP-073) and Corona (Lux LP-104) authors for the companion lattice-family threshold constructions whose architectural pattern Magnetar inherits.
- Cosmin and Khovratovich [21] for the earliest dedicated treatment of threshold SPHINCS+, whose structural pattern Magnetar refines.

10 Conclusion

10.1 What Magnetar v0.2.0 is

Magnetar v0.2.0 is the production library implementation of a threshold FIPS 205 SLH-DSA construction at Tier B in the Lux Lytle tier discipline:

- **Reference implementation.** ~ 3992 lines of Go across 12 production files plus 9 test files (§5), backed by `cloudflare/circl v1.6.3` [19] as the FIPS 205 primitive.
- **Class-N1-analog property.** Threshold signatures are byte-identical to centralized FIPS 205 SIGNDETERMINISTIC on the reconstructed master seed (Theorem 1), validated empirically by `TESTN1_BYTEEQUALITY` across three threshold configurations.
- **Deterministic KAT vectors.** Five JSON files at `vectors/` regenerable bit-identically from a clean checkout via `cmd/genkat`.
- **Polaris third-lane integration.** Joins Pulsar (M-LWE) and Corona (R-LWE) as the hash-based leg of the maximum-assurance Polaris cert profile, providing cryptographic-family diversity against any single-family algorithmic break.
- **Honest disclosure.** The v0.1 reveal-and-aggregate aggregator-TCB caveat is documented at the construction-specification, deployment-runbook, and proof-claim levels. No covert reliance on aggregator integrity; no hidden trust assumptions.

10.2 What Magnetar v0.2.0 is not

Honest characterisation of the open gates:

- **Tier A.** No mechanised proofs (EasyCrypt theory, Lean bridges, Jasmin sources) have shipped. The construction- equivalence theorem (1) is an algorithmic claim amenable to direct mechanisation; the tEUF-CMA reduction (3) is a probabilistic claim requiring a game-hop sequence. See §4.7 for the planned artefact layout.
- **Independent audit.** No external cryptographer sign-off has been performed. BLOCKERS.md BLK-9 tracks the review path; the scope includes the security argument, the FIPS 205 conformance trace, and the constant-time posture.
- **Constant-time empirical verification.** A `dudect` sweep at $\geq 10^9$ samples across the sign and combine hot paths is the outstanding side-channel deliverable (BLOCKERS.md BLK-7).
- **Full MPC threshold.** v0.1 is reveal-and-aggregate; the aggregator process is in the TCB for the brief COMBINE window. v0.2 (the research direction of §8) targets a per-signature MPC over the SLH-DSA hash tree, but the construction is nontrivial because SLH-DSA has no linear share-aggregation identity.

- **ML-KEM-768 envelope wrapping for DKG.** v0.1 DKG envelopes are plaintext (a passive network observer can see committee shares being distributed). v0.2 will wrap each envelope under ML-KEM-768 to the recipient’s long-term identity public key, matching Pulsar’s BLOCKERS.md CR-8 pattern.
- **Full 16-document NIST MPTC submission package.** The v0.2.0 scaffold under docs/mptc/ mirrors the canonical structure but omits the actual proof artefacts and audit reports that a Category-1 submission would require. BLOCKERS.md BLK-8.

10.3 Path to Tier A

The Tier A path is mechanically structured. In approximate order of dependency:

1. Mechanise Theorem 1 in Lean 4 (lowest-hanging fruit; the algorithm is purely deterministic). The proof reuses the existing `RESHARE_PRESERVES_SECRET` Lagrange identity from `lux-formal-verification-lean4` [38] modulo a $\text{GF}(257)$ port.
2. Mechanise Theorem 3 in EasyCrypt as a game-hop sequence. The hops are: Game_0 honest $\text{tEUF-CMA} \rightarrow \text{Game}_1$ simulator-replaced $\rightarrow \text{Game}_2$ SLH-DSA oracle verbatim. Bounded by `cSHAKE256 PRG/PRF` and Shamir privacy.
3. Ship the `dudect` empirical constant-time pass at $\geq 10^9$ samples for the sign and combine hot paths. Identify any data-dependent timing variance and remediate before independent review.
4. Schedule the independent cryptographer review against the full v0.1 specification (§3), the security argument (§4), the FIPS 205 conformance trace (`FIPS-TRACEABILITY.md`), and the constant-time evidence (`TRUSTED-COMPUTING-BASE.md`).
5. Address any audit findings; pin the post-audit commit as Tier A v0.3.0.

The Tier A target window is open: it depends on the audit response turnaround and any remediation cycles. Operationally, deployments running Polaris today are using Magnetar v0.2.0 at Tier B with the documented trust caveat; the safety guarantee comes from the Polaris composition (any two of three lanes intact = post-quantum finality) rather than from any single-lane assurance.

10.4 Closing

The Lux PQ transition is structured around the principle that no single primitive carries the post-quantum future of the chain. Pulsar (M-LWE), Corona (R-LWE), and Magnetar (SLH-DSA) are three independent constructions that share neither hash function family, nor secret-sharing mechanism, nor underlying mathematical assumption beyond “the standard cryptographic hash assumptions hold.” The Polaris cert profile binds all three lanes against a common transcript and accepts only when all three verify.

Magnetar’s role in this composition is the hash-based insurance leg: the lane that is intact when the entire lattice family is broken by some unforeseen algorithmic advance. It is the slowest of the three lanes, which is acceptable; the insurance does not need to be fast.

The v0.1 reveal-and-aggregate construction documented in this paper is the operationally available starting point. The v0.2 research direction (§8) is open. The Tier A mechanised proofs and independent audit are scheduled. In the meantime, Magnetar v0.2.0 ships as a library, and the deployment discipline is to run it as one of three lanes in Polaris with the aggregator hardening of `DEPLOYMENT-RUNBOOK.md` §“Aggregator hardening.”

The cryptographic primitives are not new. The composition that makes them into a hash-based threshold-SLH-DSA leg of a three-lane Polaris cert profile, with byte-equality to the unmodified FIPS 205 verifier and an honest trust-model disclosure, is Magnetar’s contribution.

References

- [1] Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors, 2015.
- [2] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Benjamin Grégoire, Adrien Koutsos, Vincent Laporte, Tiago Oliveira, and Pierre-Yves Strub. The last mile: High-assurance and high-speed cryptographic implementations. *IEEE S&P*, 2020.
- [3] Anonymous. Threshold UOV: A threshold multivariate signature. NIST MPTC submission, 2024.
- [4] Anonymous. Two-round threshold signature from algebraic one-more learning with errors. In *EUROCRYPT*. Springer, 2024.
- [5] Gilles Barthe, François Dupressoir, Benjamin Grégoire, César Kunz, Benedikt Schmidt, and Pierre-Yves Strub. EasyCrypt: A tutorial. In *Foundations of Security Analysis and Design VII*. Springer, 2013.
- [6] Anja Becker, Léo Ducas, Nicolas Gama, and Thijs Laarhoven. New directions in nearest neighbor searching with applications to lattice sieving. In *SODA*, 2016.
- [7] Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation. In *STOC*. ACM, 1988.
- [8] Daniel J. Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. SPHINCS+: Submission to the NIST post-quantum project. <https://sphincs.org>, 2022.
- [9] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. The KECCAK reference. <https://keccak.team>, 2011.
- [10] Ward Beullens. MAYO: A multivariate quadratic signature scheme. NIST PQC additional signatures, 2023.
- [11] Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter Rasmussen, and Amit Sahai. Threshold cryptosystems from threshold fully homomorphic encryption. In *CRYPTO*, pages 565–596. Springer, 2018.
- [12] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the Weil pairing. In *ASIACRYPT*. Springer, 2001.
- [13] Dan Boneh and Victor Shoup. *A Graduate Course in Applied Cryptography*. 2023.
- [14] Cecilia Boschini, Darya Kaviani, Russell W. F. Lai, Giulio Malavolta, Akira Takahashi, and Mehdi Tibouchi. Ringtail / corona: Practical two-round threshold signatures from module-LWE and module-SIS. IACR ePrint 2024/1113, 2024.
- [15] Sean Bowe. BLS12-381: New zk-SNARK elliptic curve construction. <https://electriccoin.co/blog/new-snark-curve/>, 2017.
- [16] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *FOCS*. IEEE, 2001.
- [17] Ran Canetti, Rosario Gennaro, Steven Goldfeder, Nikolaos Makriyannis, and Udi Peled. UC non-interactive, proactive, threshold ECDSA with identifiable aborts. *Cryptology ePrint Archive*, Paper 2021/060, 2021.

- [18] Andrea Celi, Rafaël del Pino, Thomas Espitau, Guilhem Niot, and Thomas Prest. Efficient threshold ML-DSA. In *USENIX Security*, 2025.
- [19] Cloudflare. circl: Cloudflare interoperable reusable cryptographic library. <https://github.com/cloudflare/circl>, 2025. v1.6.3; sign/slhdsa package implements FIPS 205.
- [20] Deirdre Connolly, Chelsea Komlo, Ian Goldberg, and Christopher A. Wood. RFC 9591: The flexible round-optimized schnorr threshold (FROST) protocol for two-round schnorr signatures. RFC Editor, 2024.
- [21] Andrei Cosmin and Dmitry Khovratovich. Threshold SPHINCS+: A hash-based threshold signature. In *SAC*. Springer, 2021.
- [22] Ivan Damgård, Claudio Orlandi, Akira Takahashi, and Mehdi Tibouchi. Two-round n -out-of- n and multi-signatures and trapdoor commitment from lattices. In *PKC*. Springer, 2021.
- [23] Rafael del Pino, Shuichi Katsumata, Mary Maller, Fabrice Mouhartem, Thomas Prest, and Mark-Henry Servera. Threshold raccoon: Practical threshold signatures from standard lattice assumptions. Eurocrypt, 2024.
- [24] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Dilithium: A lattice-based digital signature scheme. In *IACR Transactions on Cryptographic Hardware and Embedded Systems*, volume 2018, pages 238–268, 2018.
- [25] Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. FALCON: Fast-fourier lattice-based compact signatures over NTRU. <https://falcon-sign.info>, 2020.
- [26] Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game. In *STOC*. ACM, 1987.
- [27] Jonathan Katz and Yehuda Lindell. *Introduction to Modern Cryptography*. Chapman & Hall/CRC, 3rd edition, 2020.
- [28] Zach Kelling. LP-073: Pulsar: Dynamic lattice threshold signatures for permissionless validator sets. Lux Industries; `papers/lp-073-pulsar/lp-073-pulsar.tex`, 2026.
- [29] Zach Kelling. QuasarCert soundness: Canonical formal specification, Quasar 3.0. Lux Industries; `lux/proofs/quasar-cert-soundness.tex`, 2026.
- [30] Zach Kelling and Vishnu Seesahai. LP-104: Corona / ringtail: Lattice-based post-quantum threshold signatures for blockchain consensus. Lux Industries; `papers/lux-corona-pq/lux-ringtail-pq.tex`, 2026.
- [31] John Kelsey, Shu jen Chang, and Ray Perlner. NIST SP 800-185: SHA-3 derived functions: cSHAKE, KMAC, TupleHash and ParallelHash. <https://csrc.nist.gov/publications/detail/sp/800-185/final>, 2016.
- [32] Chelsea Komlo and Ian Goldberg. FROST: Flexible round-optimized Schnorr threshold signatures. In *SAC*, 2020.
- [33] Chelsea Komlo and Ian Goldberg. FROST: Flexible round-optimized schnorr threshold signatures. RFC 9591, 2024.

- [34] Thijs Laarhoven. Sieving for shortest vectors in lattices using angular locality-sensitive hashing. In *CRYPTO*, 2015.
- [35] Lux Core Team. LP-020: Quasar consensus 3.0. <https://github.com/luxfi/lps/blob/main/LP-020-quasar-consensus.md>, 2026.
- [36] Lux Core Team. LP-075: BLS aggregate signatures for warp messaging. <https://github.com/luxfi/lps/blob/main/LP-075-bls-aggregate.md>, 2026.
- [37] Lux Core Team. Magnetar: Threshold FIPS 205 SLH-DSA (go canonical). <https://github.com/luxfi/magnetar>, 2026. Tag v0.2.0; commits 55a7c96, 8ed01a3, 89301f0, 572147c, db6c252.
- [38] Lux Industries. Lean 4 cryptographic proof tree (lux-formal-verification-lean4). <https://github.com/luxfi/proofs/tree/main/lean/Crypto>, 2026.
- [39] Lux Industries. LP-103: Lens — curve-based threshold signatures with dynamic resharing. Lux Improvement Proposal, 2026.
- [40] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. *Journal of the ACM*, 60(6):1–35, 2013.
- [41] NIST. FIPS 203: Module-lattice-based key-encapsulation mechanism standard. <https://csrc.nist.gov/pubs/fips/203/final>, 2024.
- [42] NIST. FIPS 204: Module-lattice-based digital signature standard. <https://csrc.nist.gov/pubs/fips/204/final>, 2024.
- [43] NIST. FIPS 205: Stateless hash-based digital signature standard. <https://csrc.nist.gov/pubs/fips/205/final>, 2024. Final standard published August 2024.
- [44] NIST. NIST IR 8214C: Multi-party threshold cryptography schemes. NIST Internal Report (Draft), 2024.
- [45] NIST Multi-Party Threshold Cryptography Project. Hermine: Threshold lattice signatures with DKG and proactive refresh. NIST MPTC presentation, 2025.
- [46] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM*, 56(6):1–40, 2009.
- [47] Oscar Reparaz, Josep Balasch, and Ingrid Verbauwhede. Dude, is my code constant time? Design, Automation & Test in Europe, 2017.
- [48] Vishnu J. Seesahai and Zach Kelling. LSS: A pragmatic framework for dynamic and resilient threshold signatures with live secret resharing. Lux Industries, Inc., 2025.
- [49] Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.

Reproducibility

Source code. github.com/luxfi/magnetar at tag v0.2.0 (Tier B graduation commit 89301f0; deterministic KAT generator commit 8ed01a3; v0.1 reference implementation commit 55a7c96; submission-package mirror commits 572147c and db6c252).

Build. `cd ref/go && GOWORK=off go build ./... && GOWORK=off go test -short ./pkg/magnetar/.` The race-detector (`-race`) is supported but skips the SLH-DSA heavy paths via the `raceEnabled` build-tag pattern; `race-on` runs cover the cheap unit tests (shamir, transcript, types).

Hardware tested. Apple Silicon M2 Pro (12-core), macOS 15.4, Go 1.26.3 darwin/arm64. Linux x86_64 reference timings will land in the v0.3 evaluation appendix.

Dependencies. github.com/cloudflare/circl v1.6.3 (FIPS 205 SLH-DSA reference, BSD-3-Clause) and golang.org/x/crypto v0.32.0 (cSHAKE256 over FIPS 202 / NIST SP 800-185). No private dependencies.

Standards referenced. FIPS 205 (SLH-DSA), FIPS 202 (SHA-3 / SHAKE / cSHAKE), NIST SP 800-185 (KMAC / TupleHash / cSHAKE customisation), NIST IR 8214C [44] (Multi-Party Threshold Cryptography scope).

Companion documents. `SPEC.md` (normative construction), `DEPLOYMENT-RUNBOOK.md` (aggregator-TCB operator disclosure), `BLOCKERS.md` (Tier B $\rightarrow$ A path), `SUBMISSION.md` (NIST MPTC scaffold), `TRUSTED-COMPUTING-BASE.md` (TCB enumeration), `PROOF-CLAIMS.md` (proof-roadmap inventory), `AXIOM-INVENTORY.md` (assumed-axiom enumeration), `FIPS-TRACEABILITY.md` (line-by-line conformance trace).

Contact. security@lux.network.